

International Islamic University Chittagong

Department of Computer Science & Engineering

Course Code: CSE-2321

Course Title: Data Structures

Prepared By

Mohammed Shamsul Alam
Professor,
Dept. of CSE, IIUC

Presented By

Mysha Sarin Kabisha
Assistant Lecturer,
Dept. of CSE, IIUC

Lecture – 1

Introduction & Overview on Data Structures

Data

- **Data** means value or set of values. In both the singular and plural form, this term is data. Following are some examples of data:

34

09/06/1974

ISBN 81-203-0000-0

10, 20, 30, 40, 50

- A ***data item*** refers to a single unit of values. Data items that are divided into sub items are called ***group items***; those that are not are called ***elementary items***. For example, a student's name may be divided into three sub items – first name, middle name and last name – but the roll number of a student normally be treated as a single item.

Data

- Collections of data are frequently organized into a hierarchy of **fields (attributes)**, **records (entity)** and **files (entity set)**.
- An **entity** is something that has certain **attributes** or properties which may be assigned values. For example, an employee in an organization is an entity. The possible attributes and the corresponding values for an entity is given bellow:

Entity	:	EMPLOYEE			
Attributes	:	NAME	AGE	SEX	DESIGNATION
Values	:	Mr. X	30	M	Director

Entities with similar attributes (e.g. all the employees in an organization) constitute an **entity set**.

Data

- Each attribute of an entity set has a range of values, and is called the ***domain of attribute***. Domain is the set of all possible values that could be assigned to the particular attribute. For example, in the EMPLOYEE entity, the attribute SEX has domain as {M, F}.
- Each record (entity) in a file (entity set) may contain many field (attribute) items. But the value in a certain field may uniquely determine the record in the file. Such a field K is called a ***primary key***, and the values $k_1, k_2, \dots$ in such a field are called ***keys or key values***.
- Example 1.1: Page-1.2 [Data Structures – Seymour Lipschutz]

Information

- The term **information** is used for data with its attribute(s). In other words, information can be defined as **meaningful data** or **processed data**. For example, 34 is a data but when we say that 34 is the age of a person then it is information.

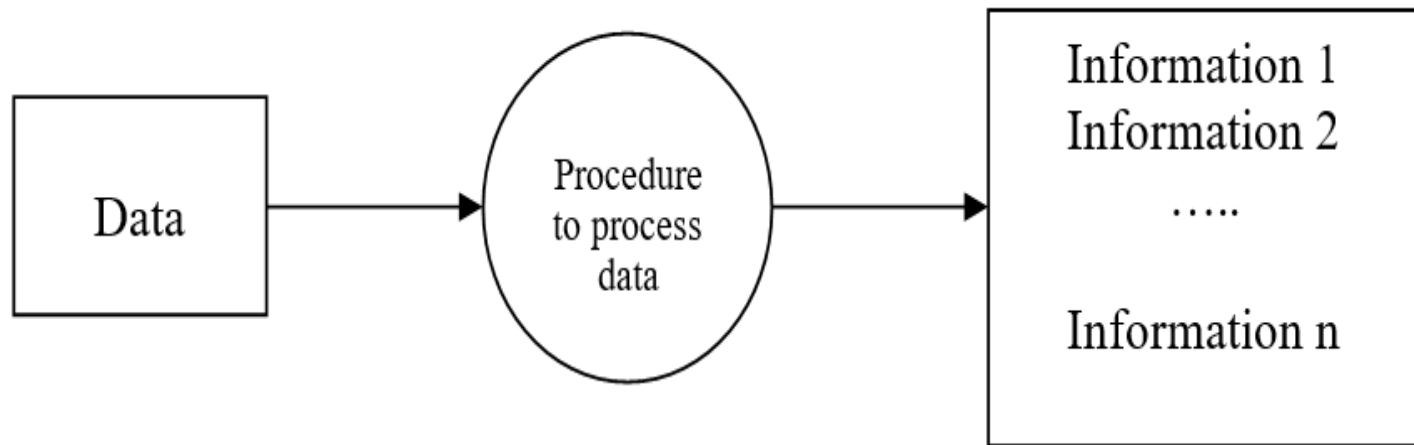


Fig: Relation between data and information

Data type

A data type is a term which refers to the kind of data that may appear in computation.

- **Built in data type:** With every programming language, there is a set of data types called built-in data types. For example, in C the built in data types are *int*, *float*, *char*, *double* etc.
- **Abstract data type:** When an application requires a special kind of data which is not available as built-in data type then it is the programmer's burden to implement his own kind of data. This programmers' own data type is termed as **abstract data type**. It is also known as *user defined data type*.

For example, we want to process dates of the form dd/mm/yy. For this, no built in data type is known in C. If a programmer wants to process dates, then an abstract data type, say **Date**, can be implemented.

- ❑ An abstract data type can be built with the help of built-in data types and other abstract data type(s) already built by the programmer. In C/C++ programmers can define their own data types by using **structure/class**.

Abstract Data Type (ADT)

Abstract Data type (ADT) is a type (or class) for objects whose behavior is defined by a set of value and a set of operations.

- The definition of ADT only mentions what operations are to be performed but not how these operations will be implemented. It does not specify how data will be organized in memory and what algorithms will be used for implementing the operations. It is called “abstract” because it gives an implementation-independent view. The process of providing only the essentials and hiding the details is known as abstraction.

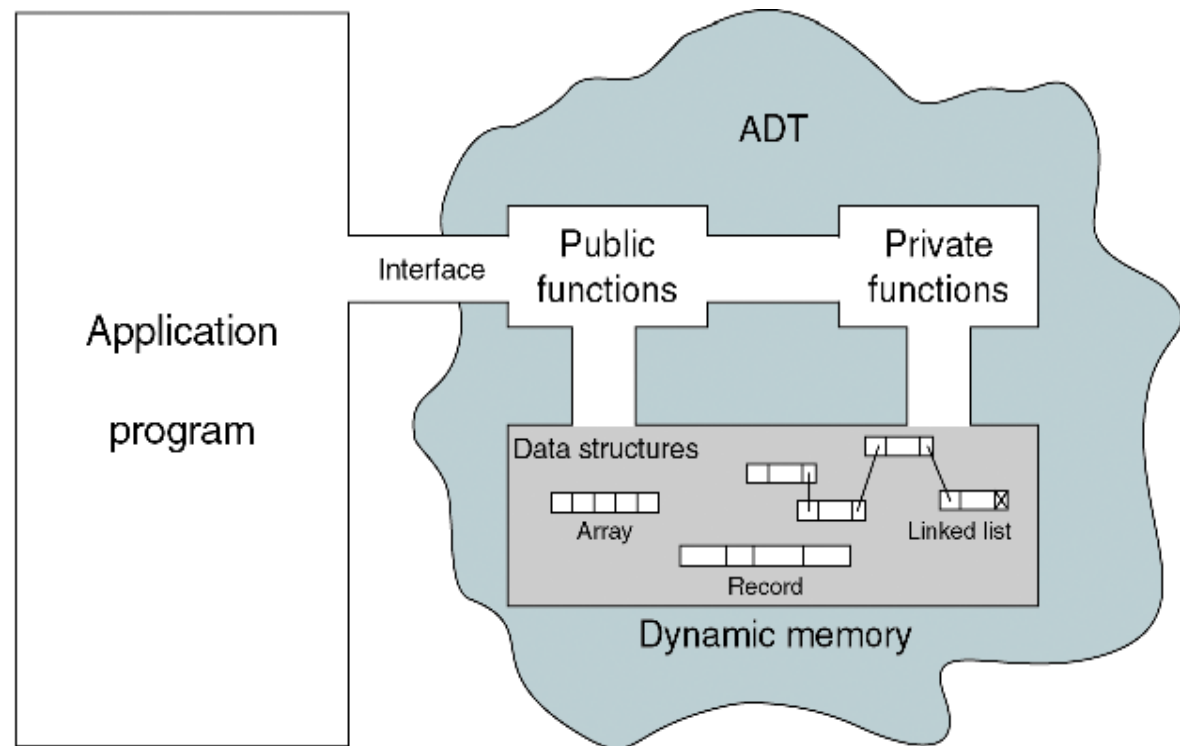


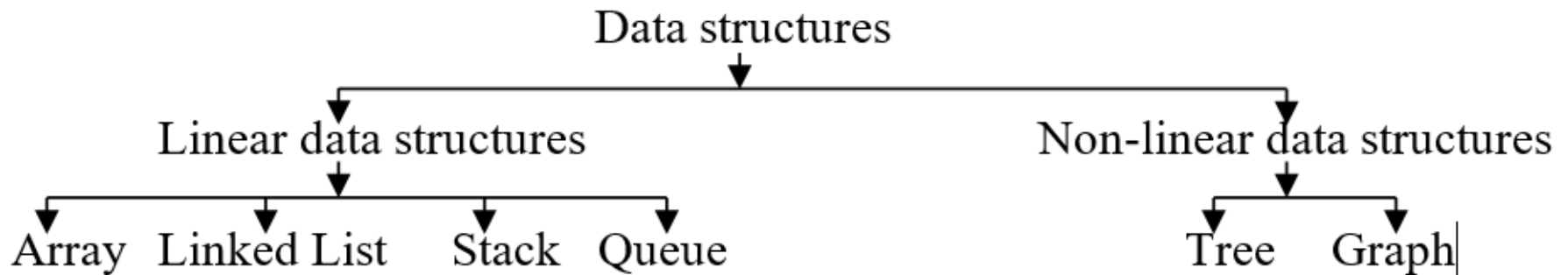
FIGURE 1-2 Abstract Data Type Model

Data structure

- Data may be organized in many different ways. The logical or mathematical model of a particular organization of data is called a ***data structure***.
- The choice of a particular data model depends on two considerations.
 - ❑ It must be rich enough in structure to mirror the actual relationships of the data in the real world.
 - ❑ The structure should be simple enough that one can effectively process the data when necessary.

Classification of Data Structures

- All the data structures can be classified into two main classes: ***linear data structures*** and ***non-linear data structures***.
- ❑ In case of linear data structures, all the elements form a sequence or maintain a linear ordering. On the other hand, no such sequence in elements, rather all the elements are distributed over a plane in case of non-linear data structures.

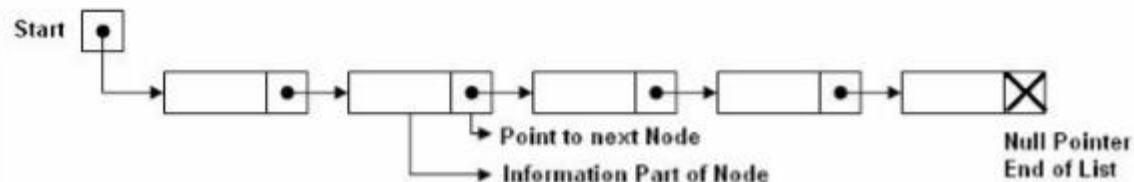


Array

- An array is a ***finite, ordered*** and collection of ***homogeneous*** data elements. Following are some examples:
 - An array of integers to store the age of all students in a class.
 - An array of strings to store the name of all students in a class.
- There are two types of array: ***one-dimensional array*** and ***multi-dimensional array***.
- By an one-dimensional (or linear) array, we mean a list of a finite number n of similar data elements referenced respectively by a set of n consecutive numbers, usually 1, 2, 3, ..., n .
- If ***A*** is the name of an 1D array, then the elements of ***A*** are denoted by ***A[1]***, ***A[2]***, ..., ***A[N]***.
- The number ***K*** in ***A[K]*** is called a *subscript* and ***A[K]*** is called a **subscripted variable**.
- We will discuss about multidimensional arrays later on.

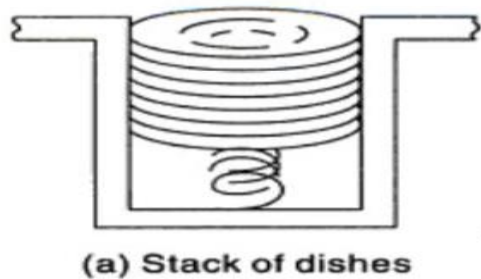
Linked list

- A **linked list** is an ordered collection of finite, homogeneous data elements, called **nodes**, where the linear order is maintained by means of **links** or **pointers**.
- In linked list, each node is divided into two parts: the first part contains the information of the element, and the second part, called the *link field* or *next pointer field*, contains the address of the next node in the list.
- There is a special pointer **START** contains the address of first node in the list. The pointer of the last node contains a special value, called **null** pointer.
- The following Fig shows a schematic diagram of a linked list with five nodes.



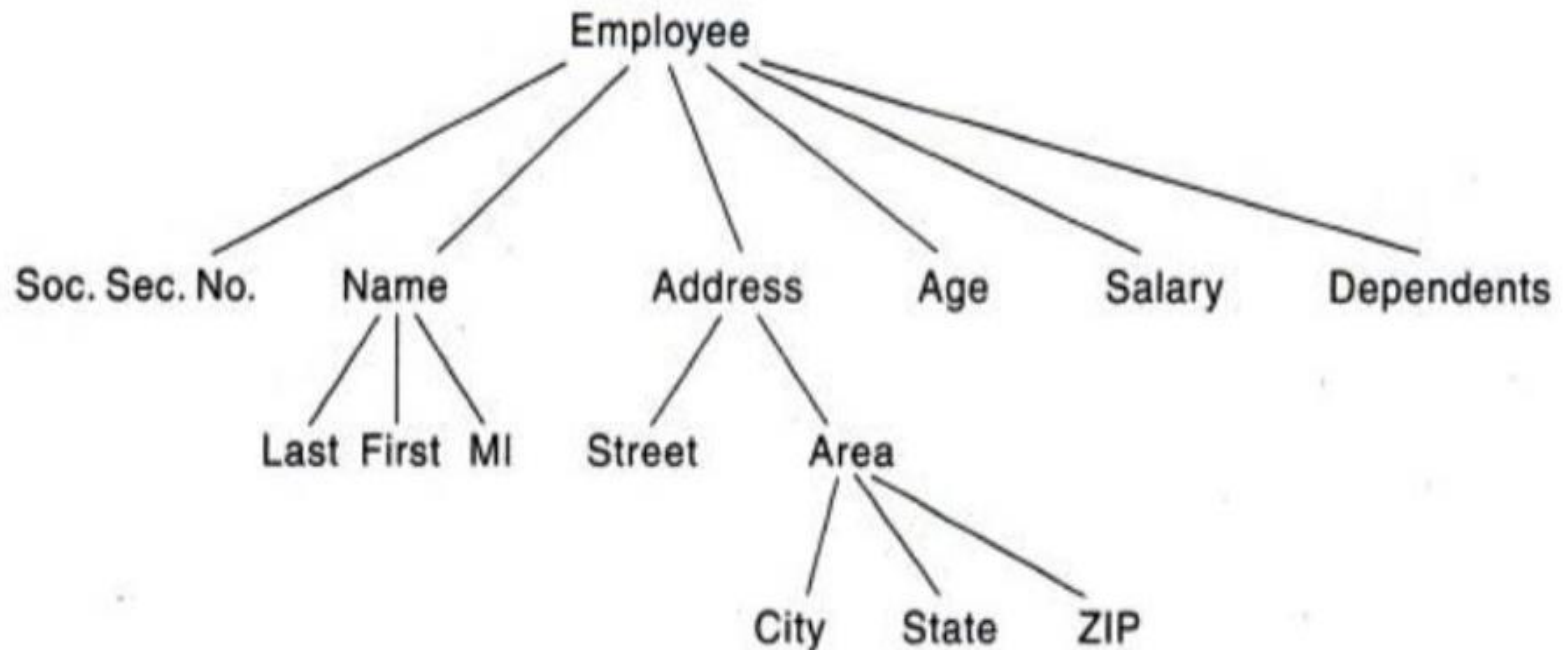
Stack & Queue

- A **stack**, also called a ***last-in first-out (LIFO)*** system, is an ordered collection of homogeneous data elements where insertion and deletion operations take place only at one end, called the ***top***. This structure is similar in its operation to a ***stack of dishes*** on a spring system, as shown in Fig (a).
- A **queue**, also called a ***first-in first-out (FIFO)*** system, is an ordered collection of homogeneous data elements where deletion operations can take place only at one end of the list, the “***front***” of the list, and insertion operations can take place only at the other end of the list, the “***rear***” of the list. This structure operates in much the same way as a line of ***people waiting at a bus stop***, as shown in Fig. (b).



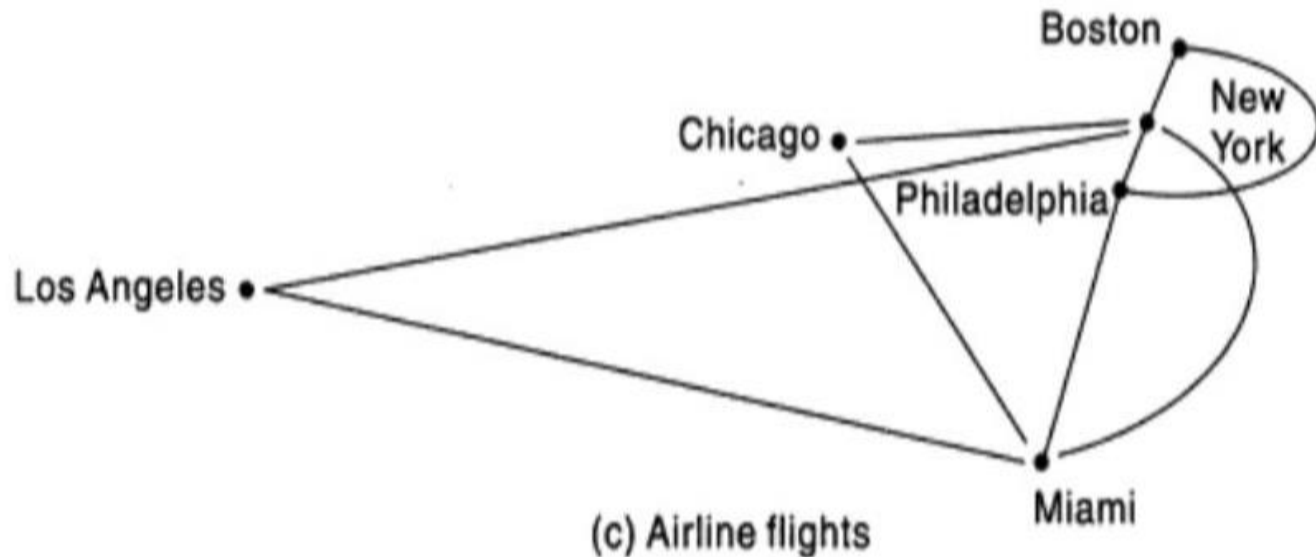
Tree

- Data frequently contain a hierarchical relationship between various elements. The data structure, which reflects this relationship, is called a **tree**. Fig. 1-7 shows an example of tree.



Graph

- Data sometimes contain a relationship between pairs of elements, which is not necessarily hierarchical in nature. For example, suppose, an airline flies only between the cities connected by the lines in Fig. The data structure, which reflects this type of relationship, is called a **graph**.



Data Structure Operations

- **Traversing:** Processing each element in the list
- **Search:** Finding the location of the element with a given value or the record with a given key.
- **Insertion:** Adding a new element to the list.
- **Deletion:** Removing an element from the list.
- **Sorting:** Arranging the elements in some type of order.
- **Merging:** Combining two sorted lists into a single list.

Complexity of an Algorithm

- An **algorithm** is a finite step-by-step list of well-defined instructions for solving a particular problem. The time and space it uses are two major measurers of the efficiency of an algorithm.
- The **complexity of an algorithm** M is a function $f(n)$ which gives the running time and/or storage space requirement of the algorithm in terms of the size n of the input data.
- ❑ Frequently, the storage space required by an algorithm is a simply a multiple of the data size n . Accordingly, the term “complexity” generally refers to the running time of the algorithm.
- **Searching Algorithms**
 - **Linear Search:**
 - **Binary Search:**

Time-Space Tradeoff of Algorithms

- The **time-space tradeoff** refers to a choice between algorithmic solutions of a data processing problem that allows one to decrease the running time of an algorithmic solution by increasing the space to store the data and vice-versa.

Home Task

- **Problem:** Write a C/C++ program to create an array of n elements and then display all the elements of the list.

International Islamic University Chittagong

Department of Computer Science & Engineering

Course Code: CSE-2321

Course Title: Data Structures

Prepared By

Mohammed Shamsul Alam
Professor,
Dept. of CSE, IIUC

Presented By

Mysha Sarin Kabisha
Assistant Lecturer,
Dept. of CSE, IIUC

Lecture – 2

Preliminaries

Mathematical Notation and Functions

Floor and Ceiling Functions

- ❑ $\lfloor x \rfloor$, called the *floor* of x , denotes the greatest integer that does not exceed x .
- ❑ $\lceil x \rceil$, called the *ceiling* of x , denotes the least integer that is not less than x .
- If x is itself an integer, then $\lfloor x \rfloor = \lceil x \rceil$; otherwise, $\lfloor x \rfloor + 1 = \lceil x \rceil$.
- $\lfloor 3.14 \rfloor = 3$; $\lceil 3.14 \rceil = 4$; $\lfloor -8.5 \rfloor = -9$; $\lceil -8.5 \rceil = -8$;
 $\lfloor 7 \rfloor = 7$; $\lceil x \rceil = 7$.

Mathematical Notation and Functions

Remainder Function

- Let k be any integer and let M be a positive integer. Then $k \pmod{M}$ will denote the integer remainder when k is divided by M .
- $k \pmod{M}$ is the unique integer r such that $k = Mq + r$ where $0 \leq r < M$.
- $25 \pmod{7} = 4$ $25 \pmod{5} = 0$ $3 \pmod{8} = 3$

Congruence

- $a \equiv b \pmod{M}$ if and only if M divides $a - b$
 $22 \equiv 2 \pmod{5}$

Mathematical Notation and Functions

Integer and Absolute Value Function

- Let x be any real number. The integer value of x , written ***INT***(x), converts x into an integer y deleting (truncating) the fractional part of the number.

$$\text{INT}(3.14) = 3$$

$$\text{INT}(-8.5) = -8$$

$$\text{INT}(7) = 7$$

- The ***absolute value*** of the real number x , written ***ABS***(x) or $|x|$, is defined as the greater of x and $-x$.

$$|-15| = 15 \quad |3| = 3 \quad |-8.5| = 8.5 \quad |8.5| = 8.5$$

$$|0| = 0$$

Mathematical Notation and Functions

Factorial Function

- $n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot (n-2) \cdot (n-1) \cdot n$
- $4! = 1 \cdot 2 \cdot 3 \cdot 4 = 24$ $0! = 1$

Fibonacci Function

- The Fibonacci sequence is as follows: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55,

Here, $F_0 = 0$ and $F_1 = 1$ and each succeeding term is the sum of two preceding terms.

Mathematical Notation and Functions

Permutation and Combination

➤ ${}^n P_r = n! / (n-r)!$

➤ ${}^n C_r = n! / r! (n-r)!$

Summation

➤ $1 + 2 + 3 + \dots + n = n(n+1) / 2$

Mathematical Notation and Functions

Exponents and Logarithms

➤ $a^m = a \cdot a \cdot \dots \cdot a$ (m times).

➤ $a^0 = 1$

➤ $b^y = x \quad y = \log_b x$

➤ $\log_2 8 = 3 \quad \log_{10} 100 = 2$

➤ *Common logarithm (base 10), Natural logarithm (base e), Binary logarithm (base 2)*

➤ *The term $\log x$ shall mean $\log_2 x$ unless otherwise specified*

Algorithmic Notation

An **algorithm** is a finite step-by-step list of well-defined instructions for solving a particular problem.

Algorithm 2.1: (Largest Element in Array) A nonempty array DATA with N numerical values is given. This algorithm finds the location LOC and the value MAX of the largest element of DATA. The variable K is used as a counter.

Step 1. [Initialize] Set $K := 1$, $LOC := 1$ and $MAX := DATA[1]$.

Step 2. [Increment counter] Set $K := K + 1$.

Step 3. [Test counter] If $K > N$, then:

Write: LOC, MAX, and Exit.

Step 4. [Compare and update] If $MAX < DATA[K]$, then:

Set $LOC := K$ and $MAX := DATA[K]$.

Step 5. [Repeat loop] Go to Step 2.

- The format for the formal presentation of an algorithm consists of *two* parts. The *first part* is a **paragraph** which tells the purpose of the algorithm, identifies the variables which occur in the algorithm and lists the input data. The *second part* of the algorithm consists of the **list of steps** that is to be executed.

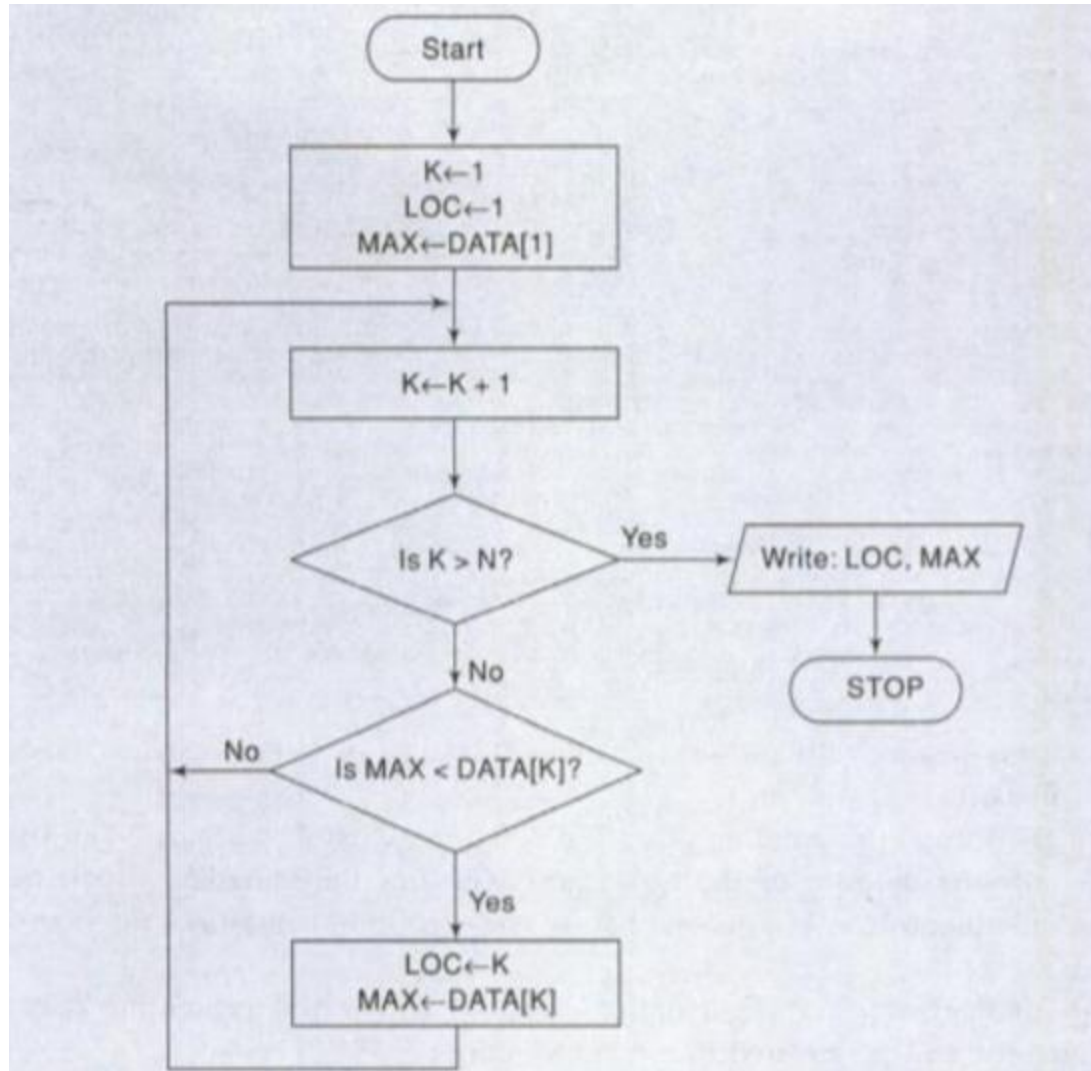
Algorithmic Notation

The following summarizes certain conventions that we will use in presenting our algorithms.

- **Steps, Control, Exit:** The steps of the algorithm are executed one after the other, beginning with Step 1, unless indicated otherwise. Control may be transferred to Step n of the algorithm by the statement “Go to Step n ”. If several statements appear in the same step, then they are executed from left to right. The algorithm is completed when the statement **Exit** is encountered.
- **Comments:** Each step may contain a comment in brackets which indicates the main purpose of the step.
- **Variable Name:** Variable names will use capital letters. Single-letter [both uppercase and lowercase] names of variables used as counters or subscripts.
- **Assignment statement:** Dots-equal notation $: =$ is used as assignment operation. Some texts use backward arrow $\leftarrow$ or the equal sign $=$ for this operation.
- **Input and Output:** For input **Read:** variables names is used. For writing, **Write:** Messages and/or variable names.

Flowchart

The *graphical representation* of an algorithm is called a **flowchart**. For the flowchart of the above algorithm see the following flowchart:



CONTROL STRUCTURES

There are **three** types of logic, or flow of control, called

- a. Sequence logic or sequential flow
- b. Selection logic or conditional flow
- c. Iteration logic or repetitive flow

Sequence Logic (Sequential Flow)

- Unless instructions are given to the contrary, the modules are executed in the obvious sequence. The sequence may be presented explicitly, by means of numbered steps, or implicitly, by the order in which the modules are written. Most processing, even of complex problems, will generally follow this elementary flow pattern.

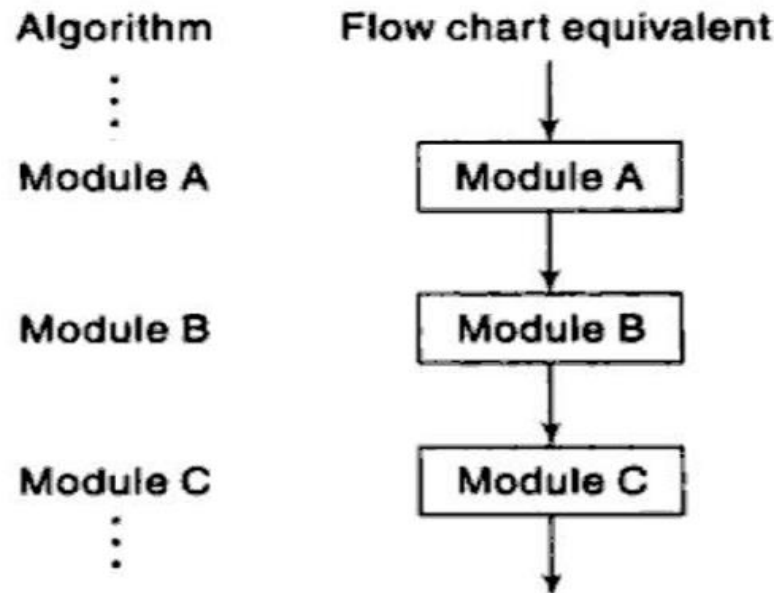


Fig. 2.3

Sequence Logic

Selection Logic (Conditional Flow)

Selection logic employs a number of conditions which lead to a selection of one out of several alternative modules. The structures which implement this logic are called conditional structures or If structures. These conditional structures fall into three types, which are discussed below.

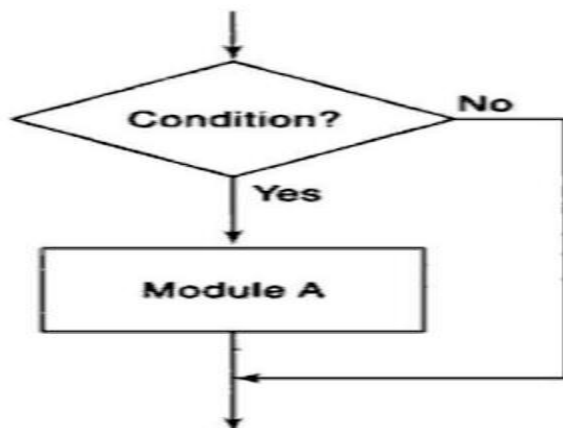
i) **Single alternative:** This structure has the form

 If condition, then:

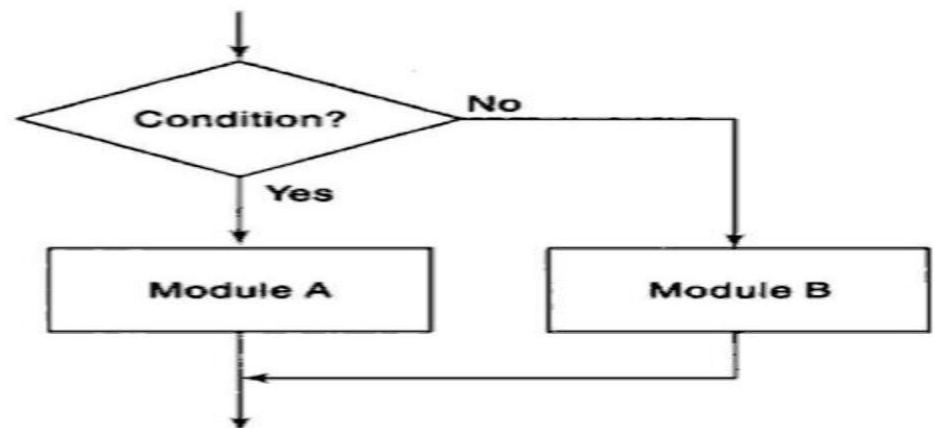
 [Module A]

 [End of If structure]

Here, if the condition is TRUE, then Module A is executed, otherwise, Module A is skipped and control transfers to the next step of the algorithm.



(a) Single alternative.



(b) Double alternative.

Selection Logic (Conditional Flow)

ii) **Double alternative:** This structure has the form

If condition, then:

[Module A]

Else:

[Module B]

[End of If structure]

In this logic, if the condition is TRUE, then Module A is executed; if it is FALSE then Module B is executed.

iii) **Multiple alternatives:** This structure has the form:

If condition (1) then:

[Module A₁]

Else if condition (2) then:

[Module A₂]

.....

Else if Condition (M) then:

[Module A_M]

Else:

[Module B]

[End of If structure]

- The logic of this structure allows only one of the modules to be executed. If the condition (1) is TRUE then Module A₁ is executed, if it is false then condition (2) is checked and if it is TRUE, then Module A₂ is executed, and so on. If all the conditions are FALSE then Module B is executed.

Quadratic Equation

Algorithm: (Quadratic Equation) This algorithm inputs the coefficients A, B, C of a quadratic equation and outputs the real solutions, if any.

Step 1. Read: A, B, C.

Step 2. Set $D := B^2 - 4AC$.

Step 3. If $D > 0$, then:

(a) Set $X1 := (-B + \sqrt{D})/2A$ and $X2 := (-B - \sqrt{D})/2A$.

(b) Write: X1, X2.

Else if $D = 0$, then:

(a) Set $X := -B/2A$.

(b) Write: 'UNIQUE SOLUTION', X.

Else:

Write: ' NO REAL SOLUTIONS'

[End of If structure]

Step 4. Exit.

❖ **Home task:** Draw the flowchart for the above algorithm.

Iteration Logic (Repetitive Flow)

There are two types of iteration logic. Each type begins with a Repeat statement and is followed by a module, called the body of the loop.

i) The ***repeat-for loop***:

Repeat for $K = R$ to S by T :

[Module]

[End of loop]

Here, R is called the *initial* value, S is the *end value* and T is the *increment*. K is called *control variable*.

ii) The ***repeat-while loop***:

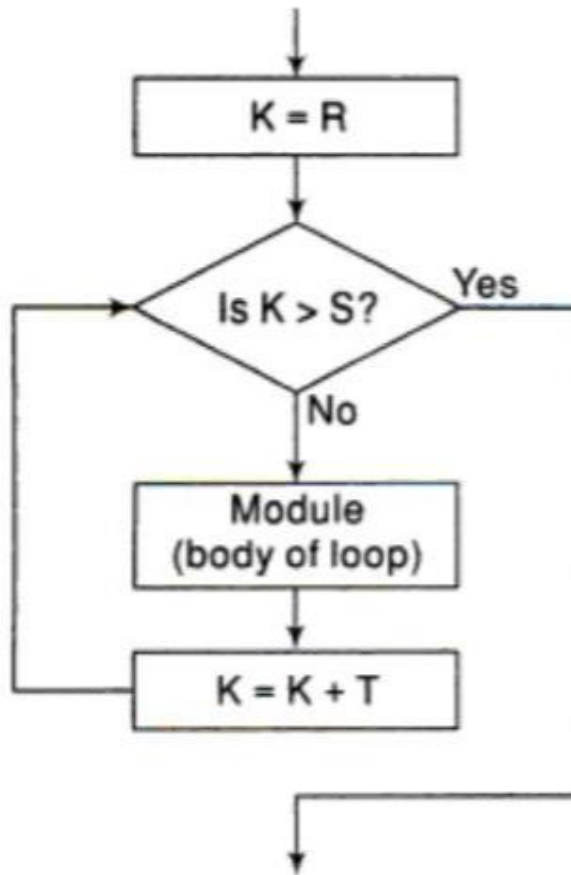
Repeat while condition:

[Module]

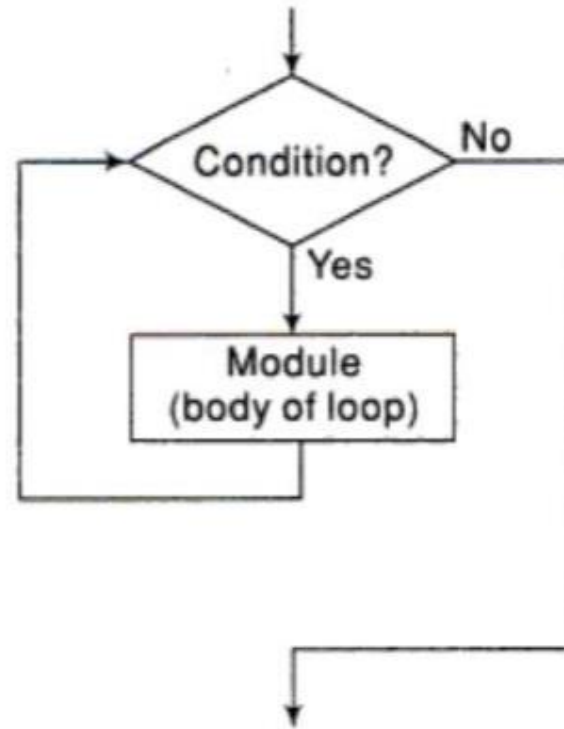
[End of loop]

Here the loop is executed until the condition is FALSE.

Iteration Logic (Repetitive Flow)



(a) Repeat-For structure.



(a) Repeat-While structure.

Largest Element in Array

Algorithm 2.3: (Largest Element in Array) given a nonempty array DATA with N numerical values, this algorithm finds the location LOC and the value MAX of the largest element of DATA.

1. [Initialize] Set $K:=1$, $LOC:=1$ and $MAX:= DATA[1]$.
2. Repeat Steps 3 and 4 while $K \leq N$:
3. If $MAX < DATA [K]$, then:
 Set $LOC: = K$ and $MAX: = DATA [K]$.
4. Set $K: = K+1$.
 [End of Step 2 loop.]
5. Write: LOC, MAX.
6. Exit.

❖ **Home task:** Draw the flowchart for the above algorithm.

SUBALGORITHMS

- A **subalgorithm** is a complete and independently defined algorithmic module which is used (or *invoked* or *called*) by some main algorithm or by some other subalgorithm. A subalgorithm receives values, called *arguments*, from an originating (calling) algorithm; performs computations; and then sends back the result to the calling algorithm. The subalgorithm is defined independently so that it may be called by many different algorithms or called at different times in the same algorithm. The relationship between an algorithm and a subalgorithm is similar to the relationship between a main program and a subprogram in a programming language.
- The main *difference* between the format of a subalgorithm and that of an algorithm is that the subalgorithm will usually have a heading of the form NAME (PAR₁, PAR₂, ... , PAR_K).
- Another difference is that the subalgorithm will have a **Return** statement rather than an **Exit** statement.

•

SUBALGORITHMS

Example 2.9

The following function subalgorithm MEAN finds the average AVE of three numbers A, B and C.

Function 2.5: MEAN(A, B, C)

1. Set $AVE := (A + B + C)/3$.
2. Return(AVE).

Note that MEAN is the name of the subalgorithm and A, B and C are the parameters. The Return statement includes, in parentheses, the variable AVE, whose value is returned to the calling program.

The subalgorithm MEAN is invoked by an algorithm in the same way as a function subprogram is invoked by a calling program. For example, suppose an algorithm contains the statement

Set $TEST := MEAN(T_1, T_2, T_3)$

where T_1 , T_2 and T_3 are test scores. The argument values T_1 , T_2 and T_3 are transferred to the parameters A, B, C in the subalgorithm, the subalgorithm MEAN is executed, and then the value of AVE is returned to the program and replaces $MEAN(T_1, T_2, T_3)$ in the statement. Hence the average of T_1 , T_2 and T_3 is assigned to TEST.

SUBALGORITHMS

Example 2.10

The following procedure SWITCH interchanges the values of AAA and BBB.

Procedure 2.6: SWITCH(AAA, BBB)

1. Set TEMP := AAA, AAA := BBB and BBB := TEMP.
2. Return.

The procedure is invoked by means of a Call statement. For example, the Call statement

Call SWITCH(BEG, AUX)

has the net effect of interchanging the values of BEG and AUX. Specifically, when the procedure SWITCH is invoked, the argument of BEG and AUX are transferred to the parameters AAA and BBB, respectively; the procedure is executed, which interchanges the values of AAA and BBB; and then the new values of AAA and BBB are transferred back to BEG and AUX, respectively.

Home Task

Problem:

1. Write a program to calculate the roots of the quadratic equation $ax^2 + bx + c = 0$ where a , b and c are known.
2. Write a program to find the largest number from a given list of integers.

International Islamic University Chittagong

Department of Computer Science & Engineering

Course Code: CSE-2321

Course Title: Data Structures

Prepared By

Mohammed Shamsul Alam
Professor,
Dept. of CSE, IIUC

Presented By

Mysha Sarin Kabisha
Assistant Lecturer,
Dept. of CSE, IIUC

Lecture – 3

Arrays

Linear Arrays

A **linear array** is a list of a *finite* number n of *homogeneous* data elements (i.e., data elements of the same type) such that:

- a) The elements of the array are *referenced* respectively by an *index set* consisting of n *consecutive numbers*.
 - b) The elements of the array are *stored* respectively in *successive memory locations*.
- The number n of elements is called the *length* or *size* of the array. If not explicitly stated, we will assume the index set consists of the integers 1, 2, N . In general the length or the number of data elements can be obtained from the index set by the formula

$$\text{Length} = \text{UB} - \text{LB} + 1$$

Where **UB** is the largest index, called the *upper bound* and **LB** is the smallest index, called the *lower bound* of the array.

- The elements of an array may be denoted by the bracket notation (used in Pascal and C/C++)

$$A[1], A[2], A[3], \dots, A[n]$$

The number **K** in **A[K]** is called a *subscript* or an *index* and **A[K]** is called a *subscripted variable*.

Linear Arrays

- *Example 4.1:* Let DATA be a 6-element array of integers such that
DATA[1] = 247 DATA[2] = 56 DATA[3] = 429 DATA[4] = 135
DATA[5] = 87 DATA[6] = 156

- The array DATA is frequently pictured as in

DATA

1	247
2	56
3	429
4	135
5	87
6	156

(a)

DATA

247	56	429	135	87	156
1	2	3	4	5	6

(b)

Representation of Linear Arrays in Memory

Let **LA** be a linear array in the memory of the computer. We know, the memory of the computer is simply a sequence of addressed locations. Let us use the notation:

LOC (LA[K]) = address of the element LA[K] of the array LA

As previously noted, the elements of LA are stored in successive memory cells. Accordingly, the computer does not need to keep track of the address of every elements of LA, but needs to keep track only of the address of the first element of LA, denoted by **BASE (LA)** and called the **based address** of LA. Using this address BASE (LA), the computer calculates the address of any element of LA by the following formula:

$$\mathbf{LOC\ (LA[K]) = BASE(LA) + w\ (K - LB)}$$

Where **w** is the number of words per memory cell for the array LA.

- Note that the time to calculate LOC (LA[K]) is essentially the same for any value of **K**. Furthermore, given any subscript K, one can locate and access the content of LA[K] without scanning any other element of LA.

Representation of Linear Arrays in Memory

Example 4.4

Consider the array AUTO in Example 4.1(b), which records the number of automobiles sold each year from 1932 through 1984. Suppose AUTO appears in memory as pictured in Fig. 4.4. That is, $Base(AUTO) = 200$, and $w = 4$ words per memory cell for AUTO. Then

$$LOC(AUTO[1932]) = 200, \quad LOC(AUTO[1933]) = 204, \quad LOC(AUTO[1934]) = 208, \quad \dots$$

The address of the array element for the year $K = 1965$ can be obtained by using Eq. (4.2):

$$\begin{aligned} LOC(AUTO[1965]) &= Base(AUTO) + w(1965 - \text{lower bound}) \\ &= 200 + 4(1965 - 1932) = 332 \end{aligned}$$

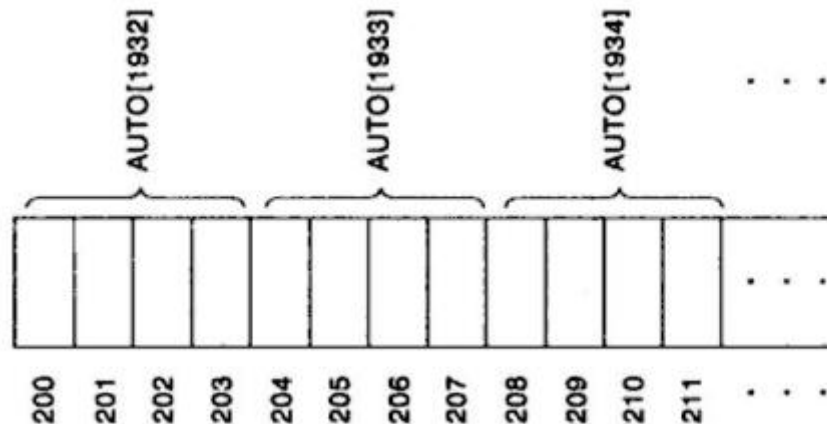


Fig. 4.4

Traversing Linear Arrays

Let **A** be a collection of data elements stored in the memory of the computer. Suppose we want to print the contents of each element of **A** or suppose we want to count the number of elements of **A** with the given property. This can be accomplished by traversing **A**, by accessing and processing (frequently called *visiting*) each element of **A** exactly once.

(Traversing a Linear array) Here LA is a linear array with lower bound LB and upper bound UB. This algorithm traverses LA applying an operation PROCESS to each element of LA.

1. Set $K := LB$
2. Repeat Steps 3 and 4 while $K \leq UB$
 Apply PROCESS to LA[K]
3. Set $K := K+1$
 [End of Step 2 Loop]
4. Exit.

□ We also state an alternative form of the algorithm which uses a repeat-for loop instead of the repeat-while loop.

(Traversing a Linear array) This algorithm traverses linear array LA with lower bound LB and upper bound UB.

1. Repeat for $K = LB$ to UB
 Apply PROCESS to LA[K]
2. Exit.

Traversing Linear Arrays

- ❑ *Caution:* the operation PROCESS in the traversal algorithm may use certain variables which must be initialized before PROCESS is applied to any of the elements in the array. Accordingly the algorithm may need to be processed by such an initialized step.
- ❑ *Example:* Consider the array AUTO, which records the number of automobiles sold each year from 2000 through 2019. Each of the following modules, which carry out the given operation, involves traversing AUTO.
 - a. **Find the number NUM of years during which more than 300 automobiles were sold.**
 - 1. Set NUM:=0.
 - 2. Repeat for K =2000 to 2019:
 - If AUTO[K] > 300, then: Set NUM:-NUM+1.
 - 3. Return.
 - b. **Print each year and the number of automobiles sold in that year.**
 - 1. Repeat for K=2000 to 2019:
 - Write: K, AUTO[K].
 - 2. Return.

Sessional: Write a program to create an array of **n** elements and then separately write the **odd** and **even** elements of the list.

Inserting into a Linear Array

Let **A** be a collection of data elements in the memory of the computer. “**Inserting**” refers to the operation of adding another element to the collection **A**.

Inserting an element at the “end” of a linear array can be easily done provided the memory space allocated for the array is large enough to accommodate the additional element. On the other hand, suppose we need to insert an element in the middle of the array. Then, on the average, half of the elements must be moved downward to new locations to accommodate the new element and keep the order of the other elements.

(Inserting into a Linear Array) INSERT (LA, N, K, ITEM)

Here **LA** is a linear array with **N** elements and **K** is a positive integer such that $K \leq N$. This algorithm inserts an element **ITEM** into the **Kth** position in **LA**.

1. Set $J := N$
2. Repeat Steps 3 and 4 while $J \geq K$
3. Set $LA [J+1] := LA [J]$
4. Set $J := J - 1$
5. Set $LA [K] := ITEM$
6. Set $N := N + 1$
7. Exit.

Deleting from a Linear Array

Let **A** be a collection of data elements in the memory of the computer. “**Deleting**” refers to the operation of removing one of the elements from **A**.

Deleting an element at the "end" of an array presents no difficulties, but deleting an element somewhere in the middle of the array would require that each subsequent element be moved one location upward in order to "fill up" the array.

(Deleting from a Linear Array) DELETE (LA, N, K, ITEM)

Here **LA** is a linear array with **N** elements and **K** is a positive integer such that $K \leq N$. This algorithm deletes the **Kth** element from **LA**.

1. Set $ITEM := LA[K]$
2. Repeat for $J = K$ to $N - 1$:
 Set $LA[J] := LA[J + 1]$
3. Set $N := N - 1$.
4. Exit.

Sessional:

1. Write a program to create an array of n elements and then insert an element to the list.
2. Write a program to create an array of n elements and then delete an element from the list.

Sorting: Bubble Sort

Let A be a list of n numbers. **Sorting** A refers to the operation of rearranging the elements of A so they are in increasing order, i.e., so that

$$A[1] < A[2] < A[3] < \dots < A[N]$$

For example, suppose A originally is the list

8, 4, 19, 2, 7, 13, 5, 16

After sorting, A is the list

2, 4, 5, 7, 3, 13, 16, 19.

(Bubble Sort) BUBBLE ($DATA, N$)

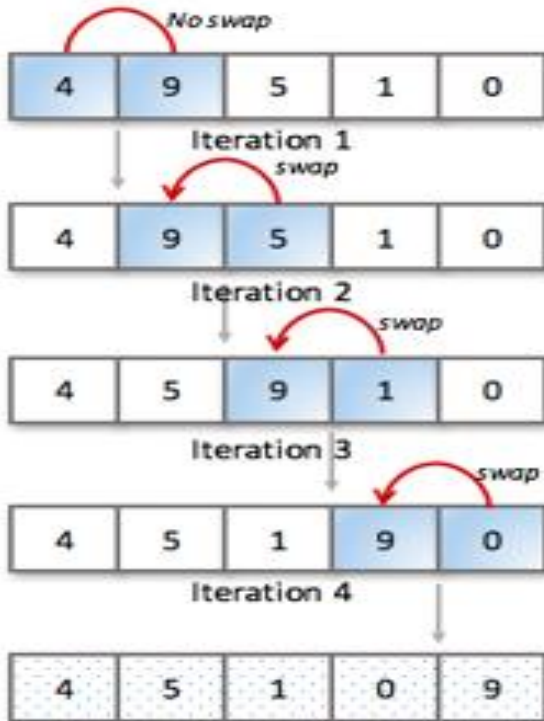
Here **DATA** is an array with **N** elements. This algorithm sorts the elements in **DATA**.

1. Repeat Steps 2 and 3 for $K = 1$ to $N - 1$
2. Set $PTR := 1$
3. Repeat while $PTR \leq N - K$:
 - (a) If $DATA[PTR] > DATA[PTR+1]$, then:
Interchange $DATA[PTR]$ and $DATA[PTR+1]$
 - (b) Set $PTR := PTR+1$
4. Exit.

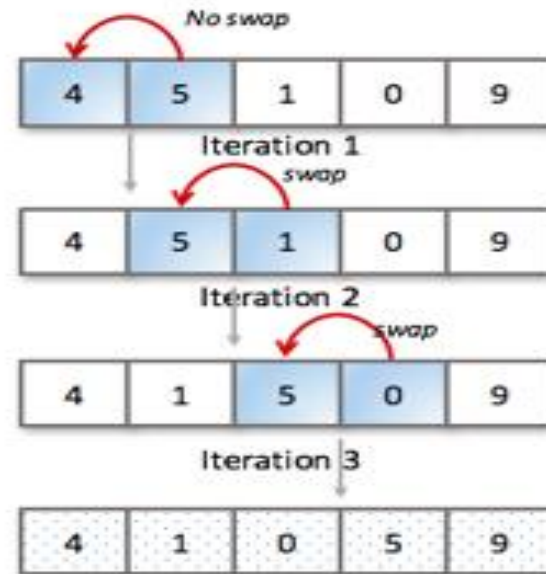
Example of Bubble Sort



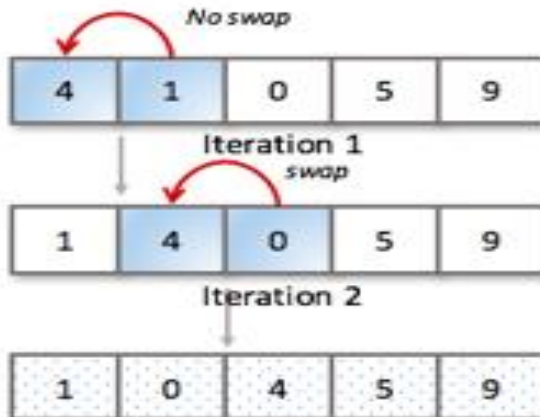
Step 1



Step 2



Step 3



Step 4



Sorting: Bubble Sort

Example 4.7

Suppose the following numbers are stored in an array A:

32, 51, 27, 85, 66, 23, 13, 57

We apply the bubble sort to the array A. We discuss each pass separately.

Pass 1. We have the following comparisons:

(a) Compare A_1 and A_2 . Since $32 < 51$, the list is not altered.

(b) Compare A_2 and A_3 . Since $51 > 27$, interchange 51 and 27 as follows:

32, (27), (51), 85, 66, 23, 13, 57

(c) Compare A_3 and A_4 . Since $51 < 85$, the list is not altered.

(d) Compare A_4 and A_5 . Since $85 > 66$, interchange 85 and 66 as follows:

32, 27, 51, (66), (85), 23, 13, 57

(e) Compare A_5 and A_6 . Since $85 > 23$, interchange 85 and 23 as follows:

32, 27, 51, 66, (23), (85), 13, 57

(f) Compare A_6 and A_7 . Since $85 > 13$, interchange 85 and 13 to yield:

32, 27, 51, 66, 23, (13), (85), 57

(g) Compare A_7 and A_8 . Since $85 > 57$, interchange 85 and 57 to yield:

32, 27, 51, 66, 23, 13, (57), (85)

At the end of this first pass, the largest number, 85, has moved to the last position. However, the rest of the numbers are not sorted, even though some of them have changed their positions.

For the remainder of the passes, we show only the interchanges.

Pass 2. (27), (33), 51, 66, 23, 13, 57, 85

27, 33, 51, (23), (66), 13, 57, 85

27, 33, 51, 23, (13), (66), 57, 85

27, 33, 51, 23, 13, (57), (66), 85

At the end of Pass 2, the second largest number, 66, has moved its way down to the next-to-last position.

Pass 3. 27, 33, (23), (51), 13, 57, 66, 85

27, 33, 23, (13), (51), 57, 66, 85

Pass 4. 27, (23), (33), 13, 51, 57, 66, 85

27, 23, (13), (33), 51, 57, 66, 85

Pass 5. (23), (27), 13, 33, 51, 57, 66, 85

23, (13), (27), 33, 51, 57, 66, 85

Pass 6. (13), (23), 27, 33, 51, 57, 66, 85

Pass 6 actually has two comparisons, A_1 with A_2 and A_2 and A_3 . The second comparison does not involve an interchange.

Pass 7. Finally, A_1 is compared with A_2 . Since $13 < 23$, no interchange takes place.

Since the list has 8 elements; it is sorted after the seventh pass. (Observe that in this example, the list was actually sorted after the sixth pass. This condition is discussed at the end of the section.)

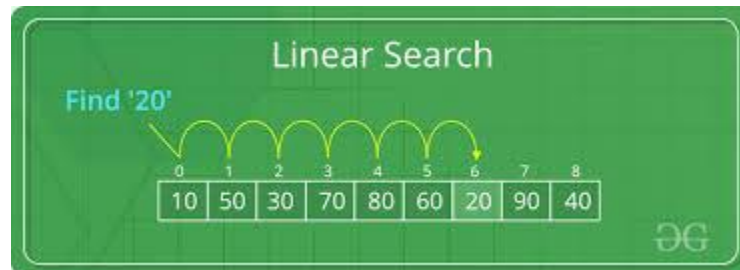
Searching

Let DATA be a collection of data elements in memory, and suppose a specific ITEM of information is given. *Searching* refers to the operation of finding the location LOC of ITEM in DATA, or printing some message that ITEM does not appear there. The search is said to be **successful** if ITEM does appear in DATA and **unsuccessful** otherwise.

There are many different searching algorithms. The algorithm that one chooses generally depends on the way the information in DATA is organized. Here we will discuss two algorithms called **linear search** and **binary search**.

Linear Search

Suppose DATA is a linear array with n elements. Given no other information about DATA, the most intuitive way to search for a given ITEM in DATA is to compare ITEM with each element of DATA one by one. That is, first we test whether $\text{DATA}[1] = \text{ITEM}$, and then we test whether $\text{DATA}[2] = \text{ITEM}$, and so on. This method, which traverses DATA sequentially to locate ITEM, is called **linear search** or **sequential search**.



Linear Search

(Linear Search) A linear array **DATA** with **N** elements and a specific **ITEM** of information are given. This algorithm finds the location **LOC** of **ITEM** in the array **DATA** or sets **LOC=0**.

1. Set $K := 1$ and $LOC := 0$
2. Repeat Steps 3 and 4 while $LOC = 0$ and $K \leq N$
3. If $ITEM = DATA[K]$, then: Set $LOC := K$
4. Set $K := K + 1$
5. If $LOC = 0$, then:
 - Write: ITEM is not in the array DATA
 - Else:
 - Write: LOC is the location of ITEM
6. Exit.

Linear Search

The following algorithm is suitable when one wants to add the element ITEM to DATA after an unsuccessful search for ITEM in DATA.

(Linear Search) LINEAR (DATA, N, ITEM, LOC)

Here DATA is a linear array with N elements, and ITEM is a given item of information. This algorithm finds the location LOC of ITEM in DATA or sets $LOC = 0$ if the search is unsuccessful.

1. [Insert ITEM at the end of DATA] Set $DATA[N + 1] := ITEM$
2. Set $LOC := 1$
3. Repeat while $DATA[LOC] \neq ITEM$:
 Set $LOC := LOC + 1$.
4. [Successful?] If $LOC = N + 1$, then: Set $LOC := 0$
5. Exit.

Remark: Here, Step-1 guarantees that the loop in step 3 must terminate.

Binary Search

Suppose DATA is an array which is sorted in increasing numerical order or, equivalently, alphabetically. Then there is an extremely efficient searching algorithm, called **binary search**, which can be used to find the location LOC of a given ITEM of information in DATA.

(Binary Search) BINARY (DATA, LB, UB, ITEM, LOC)

Here DATA is a sorted array with lower bound LB and upper bound UB and ITEM is a given item of information. The variables BEG, END and MID denote, respectively, the beginning, end and middle locations of a segment of elements of DATA. This algorithm finds the location LOC of ITEM in DATA or sets LOC=NULL.

1. Set $BEG := LB$, $END := UB$ and $MID := \text{INT}((BEG+END)/2)$
2. Repeat Steps 3 and 4 while $BEG \leq END$ and $DATA[MID] \neq ITEM$
3. If $ITEM < DATA[MID]$, then:
 Set $END := MID-1$
 Else:
 Set $BEG := MID+1$
4. Set $MID := \text{INT}((BEG+END)/2)$
5. If $DATA[MID] = ITEM$, then:
 Set $LOC := MID$
 Else:
 Set $LOC := NULL$
6. Exit.

Binary Search

Example 4.9

Let DATA be the following sorted 13-element array:

DATA: 11, 22, 30, 33, 40, 44, 55, 60, 66, 77, 80, 88, 99

We apply the binary search to DATA for different values of ITEM.

- (a) Suppose ITEM = 40. The search for ITEM in the array DATA is pictured in Fig. 4.6, where the values of DATA[BEG] and DATA[END] in each stage of the

algorithm are indicated by circles and the value of DATA[MID] by a square. Specifically, BEG, END and MID will have the following successive values:

1. Initially, BEG = 1 and END = 13. Hence

$$\text{MID} = \text{INT}[(1 + 13)/2] = 7 \quad \text{and so} \quad \text{DATA}[\text{MID}] = 55$$

2. Since $40 < 55$, END has its value changed by $\text{END} = \text{MID} - 1 = 6$. Hence

$$\text{MID} = \text{INT}[(1 + 6)/2] = 3 \quad \text{and so} \quad \text{DATA}[\text{MID}] = 30$$

3. Since $40 > 30$, BEG has its value changed by $\text{BEG} = \text{MID} + 1 = 4$. Hence

$$\text{MID} = \text{INT}[(4 + 6)/2] = 5 \quad \text{and so} \quad \text{DATA}[\text{MID}] = 40$$

We have found ITEM in location $\text{LOC} = \text{MID} = 5$.

(1) (11), 22, 30, 33, 40, 44, 55, 60, 66, 77, 80, 88, (99)

(2) (11), 22, 30, 33, 40, (44), 55, 60, 66, 77, 80, 88, 99

(3) 11, 22, 30, (33), 40, (44), 55, 60, 66, 77, 80, 88, 99 [Successful]

Fig. 4.6 Binary Search for ITEM = 40

Binary Search

(b) Suppose $ITEM = 85$. The binary search for $ITEM$ is pictured in Fig. 4.7. Here BEG , END and MID will have the following successive values:

1. Again initially, $BEG = 1$, $END = 13$, $MID = 7$ and $DATA[MID] = 55$.
2. Since $85 > 55$, BEG has its value changed by $BEG = MID + 1 = 8$. Hence

$$MID = \text{INT}[(8 + 13)/2] = 10 \quad \text{and so} \quad DATA[MID] = 77$$
3. Since $85 > 77$, BEG has its value changed by $BEG = MID + 1 = 11$. Hence

$$MID = \text{INT}[(11 + 13)/2] = 12 \quad \text{and so} \quad DATA[MID] = 88$$
4. Since $85 < 88$, END has its value changed by $END = MID - 1 = 11$. Hence

$$MID = \text{INT}[(11 + 11)/2] = 11 \quad \text{and so} \quad DATA[MID] = 80$$

(Observe that now $BEG = END = MID = 11$.)

Since $85 > 80$, BEG has its value changed by $BEG = MID + 1 = 12$. But now $BEG > END$. Hence $ITEM$ does not belong to $DATA$.

- (1) (11), 22, 30, 33, 40, 44, 55, 60, 66, 77, 80, 88, (99)
- (2) 11, 22, 30, 33, 40, 44, 55, (60), 66, (77), 80, 88, (99)
- (3) 11, 22, 30, 33, 40, 44, 55, 60, 66, 77, (80), (88), (99)
- (4) 11, 22, 30, 33, 40, 44, 55, 60, 66, 77, (80), 88, 99 [Unsuccessful]

Fig. 4.7 Binary Search for $ITEM = 85$

Remark: Whenever $ITEM$ does not appear in $DATA$, the algorithm eventually arrives at the stage that $BEG = END = MID$. Then the next step yields $END < BEG$, and control transfers to Step 5 of the algorithm.

Binary Search

Limitations of the Binary Search Algorithm

- a. The list must be sorted and
- b. One must have direct access to the middle element to any sublist.

This means that one must essentially use a sorted array to hold the data. But keeping data in a sorted array is normally very expensive when there are many insertions and deletions. Accordingly, in such situations, one may use a different data structure, such as a linked list or a binary search tree, to store the data.

Sessional:

1. Write a program to sort n numbers using Bubble Sort algorithm.
2. Write a program to search an element from a list of n numbers using Linear Search algorithm.
3. Write a program to search an element from a list of n numbers using Binary Search algorithm.

International Islamic University Chittagong

Department of Computer Science & Engineering

Course Code: CSE-2321

Course Title: Data Structures

Prepared By

Mohammed Shamsul Alam
Professor,
Dept. of CSE, IIUC

Presented By

Mysha Sarin Kabisha
Assistant Lecturer,
Dept. of CSE, IIUC

Lecture – 4

Complexity of Algorithms

Algorithms

A finite set of instructions or logic, written in order, to accomplish a certain predefined task.

- ❑ Algorithm is not the complete code or program
- ❑ It is just the core logic (solution) of a problem
- ❑ Can be expressed either as an informal high level description as pseudo code or using a flowchart.

Characteristics of an Algorithm

- ❑ Unambiguous – Algorithm should be clear and unambiguous.
- ❑ Input – An algorithm should have 0 or more well defined inputs.
- ❑ Output – An algorithm should have 1 or more well defined outputs
- ❑ Finiteness – Algorithms must terminate after a finite no. of steps.
- ❑ Feasibility – Should be feasible with the available resources.
- ❑ Independent – An algorithm should have step-by step directions which should be independent of any programming code.

Algorithm Analysis

An algorithm is said to be efficient and fast, if it takes less time to execute and consumes less memory space.

The performance of an algorithm is measured on the basis of following properties:

1. Time Complexity
2. Space Complexity

Suppose X is an algorithm and n is the size of input data, the time and space used by the Algorithm X are the two main factors which decide the efficiency of X .

- Time Factor – The time is measured by counting the number of key operations such as comparisons in sorting algorithm
- Space Factor – The space is measured by counting the maximum memory space required by the algorithm.

Complexity of Algorithms

The **complexity of an algorithm** M is the function $f(n)$ which gives the *running time* and/or *storage space* requirement of the algorithm in terms of the size n of the input data.

- Frequently, the storage space requirement by an algorithm is simply a multiple of the data size n . So, the term complexity refers to the running time of the algorithm.
- The complexity of an algorithm can be discussed in two cases:
 - a. *Worst case*: the combination of input data for which the complexity $f(n)$ is maximum.
 - b. *Average case*: the combination of input data for which the complexity $f(n)$ is not maximum and not minimum but has an expected value.

Sometimes we discuss about the *Best case* which is the combination of input data for which the complexity $f(n)$ is minimum.

Example 2.7

Suppose we are given an English short story $TEXT$, and suppose we want to search through $TEXT$ for the first occurrence of a given 3-letter word W . If W is the 3-letter word "the," then it is likely that W occurs near the beginning of $TEXT$, so $f(n)$ will be small. On the other hand, if W is the 3-letter word "zoo," then W may not appear in $TEXT$ at all, so $f(n)$ will be large.

Asymptotic Notations

The main idea of asymptotic analysis is to have a measure of efficiency of algorithms that doesn't depend on machine specific constants, and doesn't require algorithms to be implemented and time taken by programs to be compared.

Asymptotic notations are mathematical tools to represent time complexity of algorithms for asymptotic analysis.

The following 3 asymptotic notations are mostly used to represent time complexity of algorithms.

- ☐ O Notation
- ☐ Ω Notation
- ☐ θ Notation

Rate of Growth

Suppose M is an algorithm, and suppose n is the size of the input data. Clearly the complexity $f(n)$ of M increases as n increases. It is usually the rate of increase of $f(n)$ that we want to examine. This is usually done by comparing $f(n)$ with some standard function, such as

$1, \log_2 n, n, n \log_2 n, n^2, n^3, 2^n, n!$

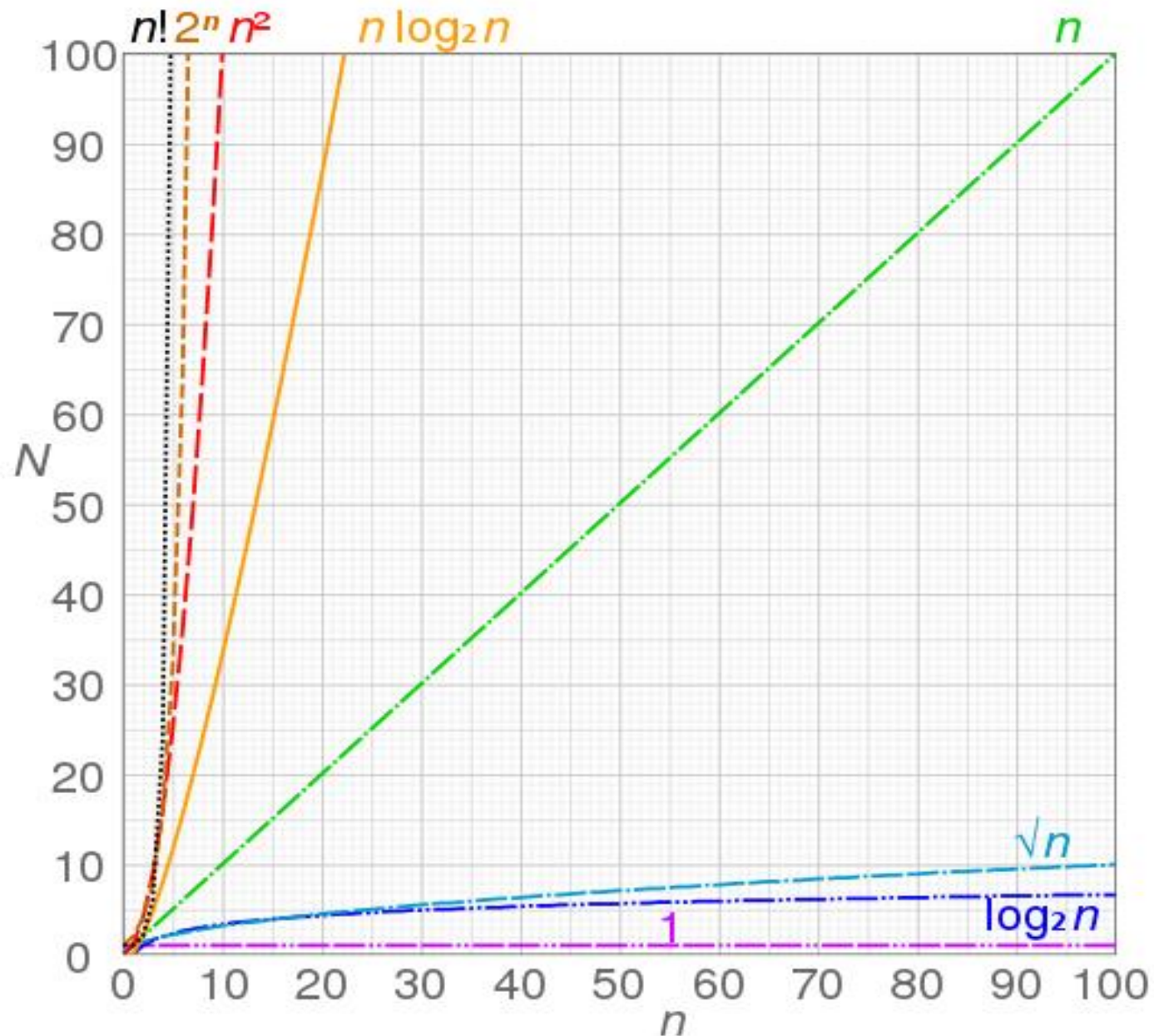
The rates of growth for these standard functions are indicated in the following Table:

$\begin{matrix} g(n) \\ n \end{matrix}$	$\log n$	n	$n \log n$	n^2	n^3	2^n
5	3	5	15	25	125	32
10	4	10	40	100	10^3	10^3
100	7	100	700	10^4	10^6	10^{30}
1000	10	10^3	10^4	10^6	10^9	10^{300}

Fig. 2.6 Rate of Growth of Standard Functions

- Observe that the functions are listed in the order of their rates of growth: the logarithmic function $\log_2 n$ grows most slowly, the exponential function 2^n grows most rapidly, and the polynomial functions n^c grow according to the exponent c .

Rate of Growth



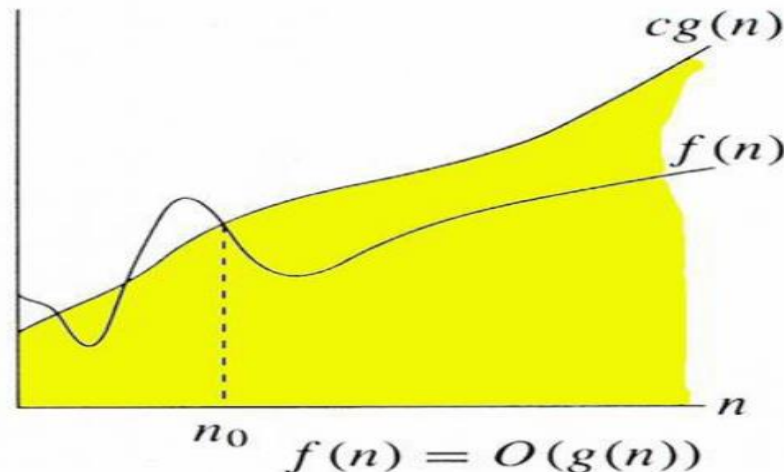
Big O Notation

It would be convenient to have a form of asymptotic notation that means "the running time grows at most this much, but it could grow more slowly." We use "big-O" notation for just such occasions.

Suppose $f(n)$ and $g(n)$ are functions defined on the positive integers with the property that $f(n)$ is bounded by some multiple of $g(n)$ for almost all n . That is, suppose there exist a positive integer n_0 and a positive number C such that, for all $n > n_0$, we have,

$$|f(n)| \leq C |g(n)|$$

Then we may write, $f(n) = O(g(n))$ which is read as " $f(n)$ is the order of $g(n)$ ".



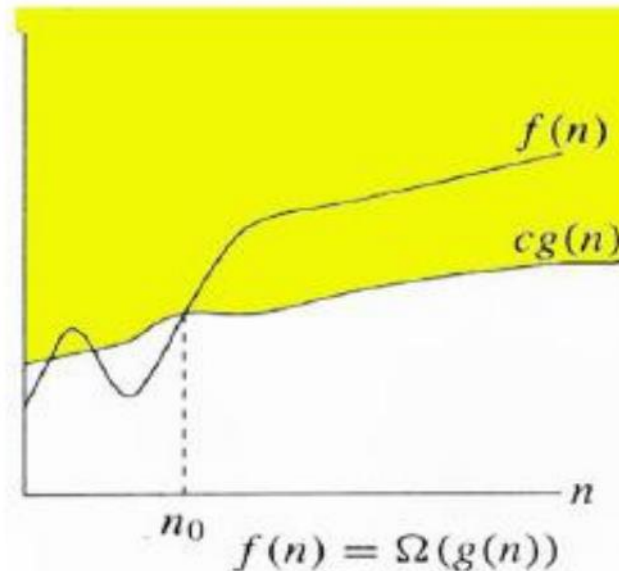
Ω (Omega) notation

Sometimes, we want to say that an algorithm takes *at least* a certain amount of time, without providing an upper bound. We use Ω notation; that's the Greek letter "omega".

Suppose $f(n)$ and $g(n)$ are functions defined on the positive integers with the property that $f(n)$ is bounded by some multiple of $g(n)$ for almost all n . That is, suppose there exist a positive integer n_0 and a positive number C such that, for all $n \geq n_0$, we have,

$$|f(n)| \geq C |g(n)|$$

Then we may write, $f(n) = \Omega(g(n))$ which is read as " f of n is the omega of g of n ".



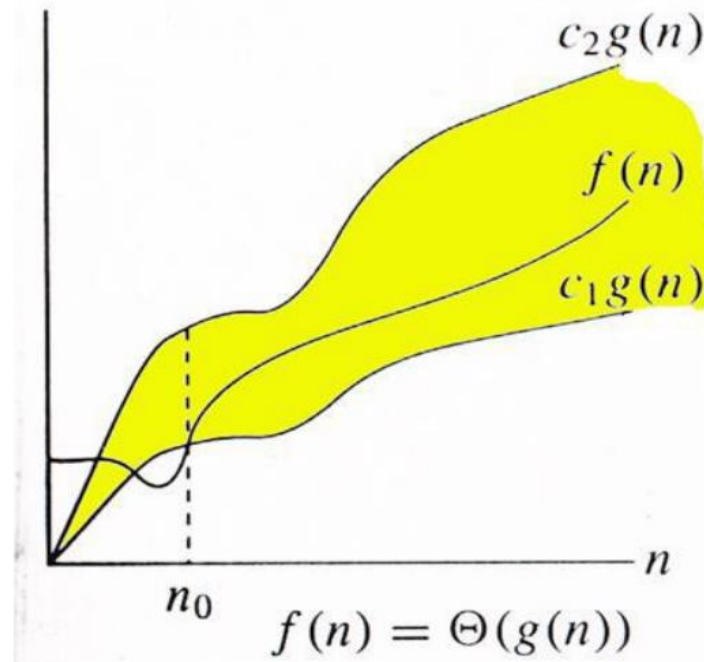
Θ (Theta) notation

We use Θ notation to asymptotically bound the growth of a running time to within constant factors above and below.

Suppose $f(n)$ and $g(n)$ are functions defined on the positive integers with the property that $f(n)$ is bounded by some multiple of $g(n)$ for almost all n . That is, suppose there exist a positive integer n_0 and two positive numbers C_1 and C_2 such that, for all $n \geq n_0$, we have,

$$C_1 |g(n)| \leq |f(n)| \leq C_2 |g(n)|$$

Then we may write, $f(n) = \Theta(g(n))$ which is read as “ f of n is theta of g of n ”.



Rate of growth

In algorithms, we focus on estimating the execution time as a function of the inputs. In particular, the main focus is how quickly does the execution time grow as a function of the inputs. For example, $f(n)=5n$ and $g(n)=100n+1000$ are equivalent functions in terms of growth with respect to n . We simply say both the functions grow linearly. Whereas, $f(n)=5n$ and $h(n)=n^2$ are not equivalent, since $h(n)$ grows quadratically.

Let's take an example. Take a look at the following pseudo-code.

```
def function1(n):
```

```
    a = 0
```

```
    for (i = 0; i < n; i += 1)
```

```
        for (j = 0; j < n; j += 1)
```

```
            a += 1
```

```
    return a
```

```
def function2(n):
```

```
    a = 0
```

```
    for (i = 0; i < n; i += 1)
```

```
        a += n
```

```
    return a
```

```
def function3(n):
```

```
    return n * n
```

Big O Notation

In the above code, function1 takes time proportional to n^2 . In particular, notice that the operation `a += 1` executes n^2 times. Similarly, function2 takes time proportional to n and function3 takes constant amount of time to run regardless of the values of n . We write the run-times of function1 as $O(n^2)$, function2 as $O(n)$ and function3 as $O(1)$.

Having a run-time of $O(n)$ means that the run-time is less than or equal to $c \cdot n$, for some constant value c (doesn't depend on n). For function2 in the above example, assuming variable assignment, addition, comparison are one operation each, we can say that number of operations being done is $< 10n$. In particular, we do approximately $2n$ assignments, $2n$ additions and n comparisons, which is a total of $5n$ operations (Here, we're treating `a += 1` as an addition operation followed by an assignment operation. Don't forget to count operations done on the loop variable `i`).

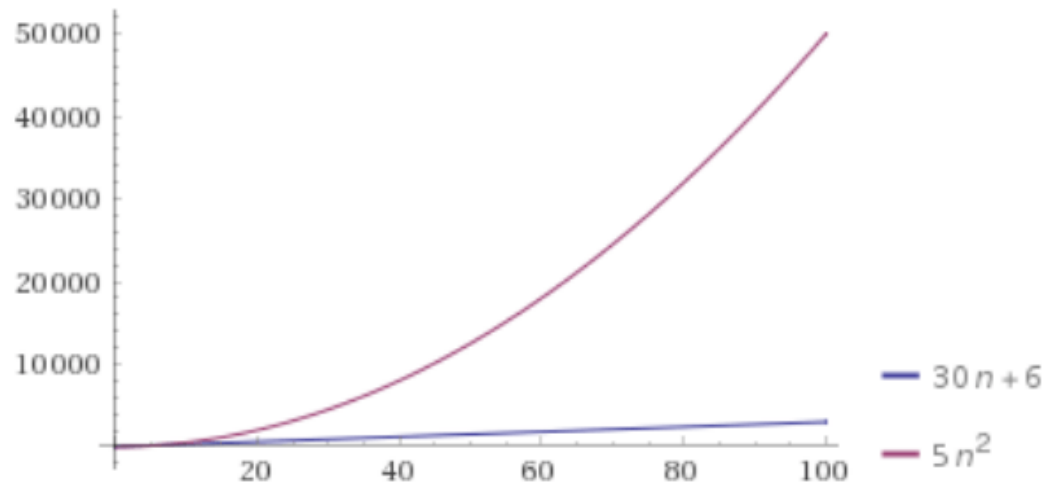
Writing a function in Big-O notation

Converting a function to big-O notation requires two simple rules.

Only consider the highest-order term. (Ignore all lower order-terms.)

Ignore any coefficients or constant factors.

Let's take an example. Suppose the total number of operations being done is $5n^2+30n+6$. In big-O notation, this function is $O(n^2)$. To get this, first we drop the lower order terms ($30n$ and the constant 6), and only keep the highest-order term $5n^2$. Next, we drop the constant factor 5 .



In our earlier example, we had $f(n)=5n=O(n)$ and $g(n)=100n+1000=O(n)$. Since both $f(n)$ and $g(n)$ are $O(n)$, we said $f(n)$ is equivalent to $g(n)$. However, since $h(n)=n^2=O(n^2)$, we concluded that $h(n)$ is not equivalent to $f(n)$.

Why do we drop the constant factors?

The big-O notation ignores constants to focus on rate of growth. In the previous example, we can see that when we have a value of n that is 4 times larger, both $f(n)$ and $g(n)$ will become 4 times larger. However, $h(n)$ will become 16 times larger. This is what we mean when we say that big-O notation focuses on how quickly does a function grow.

In practice though, we can't completely ignore constant factors. Usually all we need to keep in mind regarding constants is that if we did 10 times more operations in the part of the code executing the most number of times, then the code will take 10 times longer to execute. In the code examples we saw, this would be doing 10 times more operations inside the innermost for loop. That is, we try to roughly estimate the constant factor for the highest-order term.

However, assigning the constants meaning in an absolute sense is not feasible since there are other constants buried all over the place. For example, different operations - addition vs multiplication vs comparison - take different amounts of time to execute. Each hardware takes different amounts of time to perform different operations, and some hardwares might be capable of combining two operations into one (is a $+= 1$ one operation - increment, or two operations - addition followed by assignment).

Common algorithm complexities and their run-times

Below table lists some of the commonly occurring run-time complexities. The second column indicates what is the largest value of n for which the code would finish running within 1 second (assuming 1 billion operations per second, and constant c to be about 5-10). The third column provides examples of algorithms (many of which we'll learn in this course) with the stated run-time complexity.

1	Complexity	Max n	Examples
2	-----	-----	-----
3	$O(\log n)$	Very Large!	Binary search, Exponentiation by repeated squaring
4	$O(n)$	100,000,000	Linear search, Breadth-first search
5	$O(n \log n)$	5,000,000	Merge-sort, Quick-sort, Heap-sort, Djikstra's algo
6	$O(n^2)$	10,000	Bubble-sort
7	$O(n^3)$	400	All-pairs shortest paths
8	$O(2^n)$	25	Listing all subsets
9	$O(n!)$	11	Listing all permutations

Exercise on Big O Notation

Exercise

a.

```
sum = 0;
for( i = 0; i < n; i++)
    for( j = 0; j < n * n; j++)
        sum++;
```

Ans : $O(n^3)$

b.

```
sum = 0;
for( i = 0; i < n; i++)
    for( j = 0; j < i; j++)
        sum++;
```

Ans : $O(n^2)$

c.

```
sum = 0;
for( i = 0; i < n; i++)
    for( j = 0; j < i*i; j++)
        for( k = 0; k < j; k++)
            sum++;
```

Ans : $O(n^5)$

d.

```
sum = 0;
for( i = 0; i < n; i++)
    sum++;
    val = 1;
for( j = 0; j < n*n; j++)
    val = val * j;
```

Ans : $O(n^2)$

Space Complexity

Space complexity is the total amount of memory space used by an algorithm/program including the space of input values for execution. So to find space complexity, it is enough to calculate the space occupied by the variables used in an algorithm/program.

- But often, people confuse Space complexity with **Auxiliary space**. Auxiliary space is just a temporary or extra space and it is not the same as space complexity. In simpler terms,

Space Complexity = Auxiliary space + Space use by input values

Why do you need to calculate space complexity?

Similar to Time Complexity, Space complexity also plays a crucial role in determining the efficiency of an algorithm/program. If an algorithm takes up a lot of time, you can still wait, run/execute it to get the desired output. But, **if a program takes up a lot of memory space, the compiler will not let you run it.**

How to calculate Space Complexity of an Algorithm?

Example-1:

```
int sum(int x, int y, int z) {  
    int r = x + y + z;  
    return r;  
}
```

requires 3 units of space for the parameters and 1 for the local variable, and this never changes, so space complexity is **$O(1)$** .

Example-2:

```
int sum(int a[], int n) {  
    int r = 0;  
    for (int i = 0; i < n; ++i) {  
        r += a[i];  
    }  
    return r;  
}
```

requires N units for a, plus space for n, r and i, so it's **$O(N)$** .

Complexity of the Linear Search Algorithm

The complexity of linear search algorithm is measured by the number $f(n)$ of comparisons required to find ITEM in DATA where DATA contains n elements. Two important cases to consider are the average case and the worst case.

Worst Case: Clearly the worst case occurs when ITEM is the last element in the array DATA or is not there at all. In either situation, we have $f(n) = n$.

Accordingly, $f(n) = n = O(n)$ is the worst-case complexity of the linear search algorithm.

Average Case: Here we assume that ITEM does appear in DATA and that it is equally likely to occur at any position in the array. Accordingly, the number of comparisons can be any of the numbers 1, 2, 3, ..., n , and each number occurs with probability $p = 1/n$. Then we can write

$$\begin{aligned} f(n) &= 1 \cdot \frac{1}{n} + 2 \cdot \frac{1}{n} + \dots + n \cdot \frac{1}{n} = (1 + 2 + 3 + \dots + n) \cdot \frac{1}{n} \\ &= \frac{n(n+1)}{2} \cdot \frac{1}{n} = \frac{n+1}{2} = O(n) \end{aligned}$$

That is, the average number of comparisons required to find the location of ITEM is approximately equal to half the number of elements in the array.

Complexity of the Binary Search Algorithm

The complexity is measured by the number $f(n)$ of comparisons to locate ITEM in DATA where DATA contains n elements. Observe that each comparison reduces the sample size in half. Hence we require at most $f(n)$ comparisons to locate ITEM where

$$2^{f(n)} > n \text{ or equivalently } f(n) = \log_2 n + 1$$

That is, the running time for the worst case is approximately equal to **$\log_2 n$** . One can also show that the running time for the average case is approximately equal to the running time for the worst case.

Example 4.11

Suppose DATA contains 1 000 000 elements. Observe that

$$2^{10} = 1024 > 1000 \quad \text{and hence} \quad 2^{20} > 1000^2 = 1\,000\,000$$

Accordingly, using the binary search algorithm, one requires only about 20 comparisons to find the location of an item in a data array with 1 000 000 elements.

Complexity of the Bubble Sort Algorithm

The number of comparisons $f(n)$ in bubble sort does not depend on the primary arrangement of data. Hence, for both worst and average case in bubble sort, there is a constant number of comparisons, which depends only on n , the number of input data.

The number $f(n)$ of comparisons in the bubble sort is easily computed. Specifically, there are $n - 1$ comparisons during the first pass, which places the largest element in the last position; there are $n - 2$ comparisons in the second step, which places the second largest element in the next-to-last position; and so on. Thus

$$f(n) = (n-1) + (n-2) + \dots + 2 + 1 = n * (n-1) / 2 = n^2 / 2 + O(n) = O(n^2)$$

In other words, the time required to execute the bubble sort algorithm is proportional to n^2 , where n is the number of input items.

International Islamic University Chittagong

Department of Computer Science & Engineering

Course Code: CSE-2321

Course Title: Data Structures

Prepared By

Mohammed Shamsul Alam
Professor,
Dept. of CSE, IIUC

Presented By

Mysha Sarin Kabisha
Assistant Lecturer,
Dept. of CSE, IIUC

Lecture – 5

Multidimensional Arrays

Multidimensional Arrays

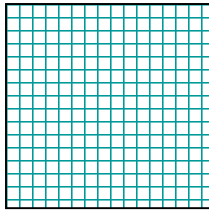
A **multi-dimensional array** is an array of arrays.

- 2-dimensional arrays are the most commonly used. They are used to store data in a tabular manner.
- Most programming languages allow two-dimensional and three-dimensional arrays, i.e., arrays where elements are referenced, respectively by two and three subscripts.
- In fact some programming languages allow the number of dimensions for an array to be as high as 7.
- In C programming an array can have two, three, or even ten or more dimensions. The maximum dimensions a C program can have depends on which compiler is being used.

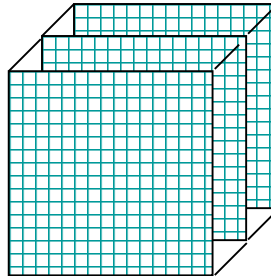
Higher-Dimensional Arrays

An array can be declared with multiple dimensions.

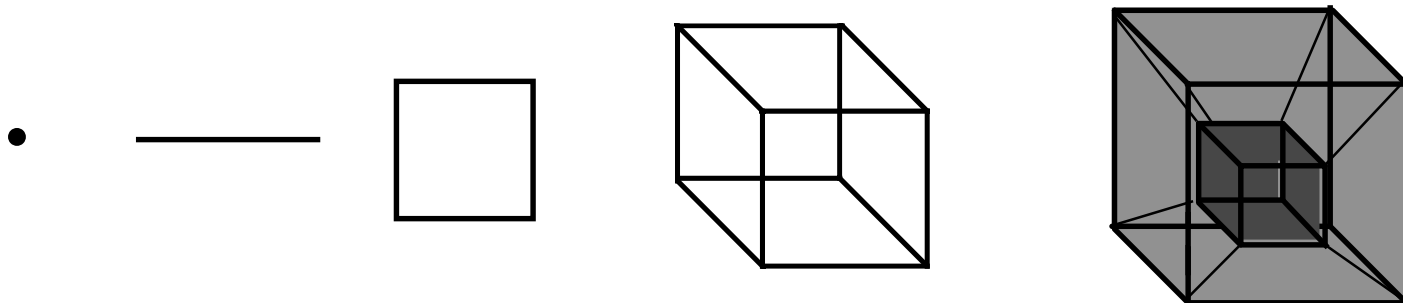
2 Dimensional



3 Dimensional



Multiple dimensions get difficult to visualize graphically.



Two-Dimensional Arrays

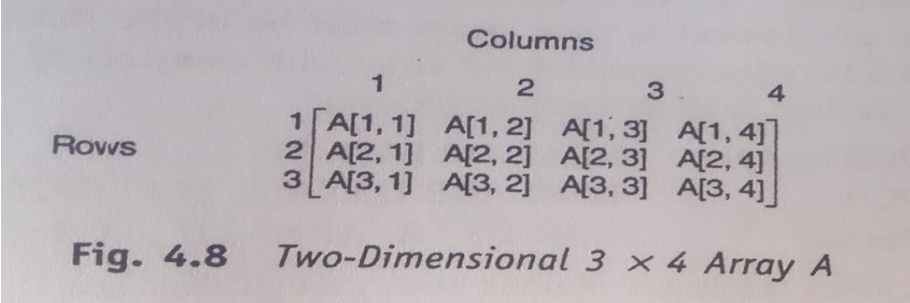
A **two-dimensional** $m \times n$ array A is a collection of $m \cdot n$ data elements such that each element is specified by a pair of integers (such as j , k), called subscripts, with the property that

$$1 \leq j \leq m \quad \text{and} \quad 1 \leq k \leq n.$$

- The element of A with first subscript j and second subscript k will be denoted by

$$A_{j,k} \quad \text{or} \quad A[j,k] \quad \text{or} \quad A[j][k]$$

- Two-dimensional arrays are called **matrices** in mathematics and **tables** in business applications; hence two-dimensional arrays are sometimes called matrix arrays.
- A standard way of drawing a two-dimensional $m \times n$ array A where the elements of A form a rectangular array with **m rows** and **n columns** and where the element $A[j,k]$ appears in row j and column k . Fig- 4.8 [page-4.19] shows a 3×4 array.



The diagram shows a 3x4 array A. The columns are labeled 1, 2, 3, 4. The rows are labeled 1, 2, 3. The elements are arranged in a grid:

	1	2	3	4
1	$A[1, 1]$	$A[1, 2]$	$A[1, 3]$	$A[1, 4]$
2	$A[2, 1]$	$A[2, 2]$	$A[2, 3]$	$A[2, 4]$
3	$A[3, 1]$	$A[3, 2]$	$A[3, 3]$	$A[3, 4]$

Fig. 4.8 Two-Dimensional 3×4 Array A

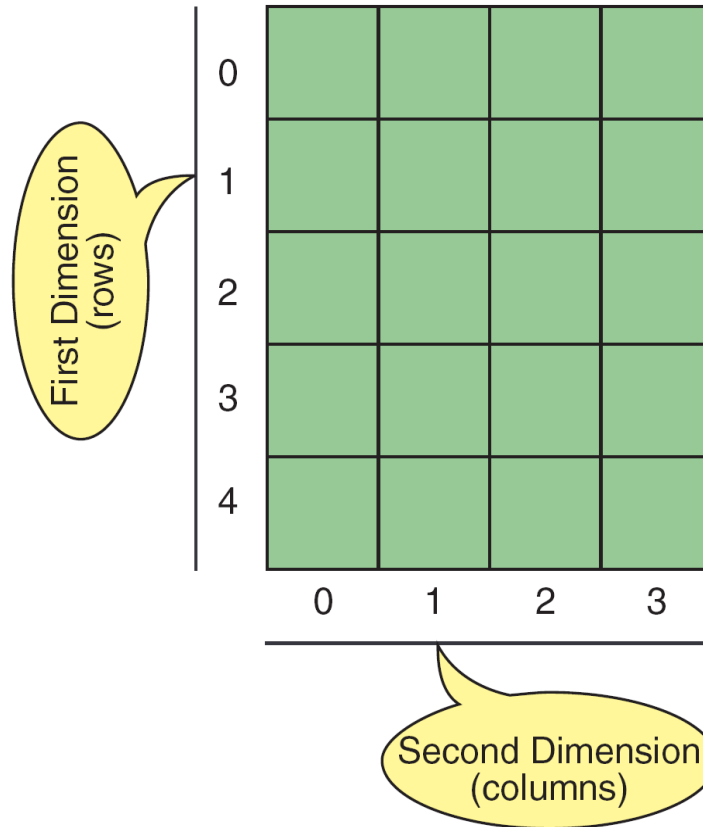


FIGURE 8-34 A Two-dimensional Array (5 x 4)

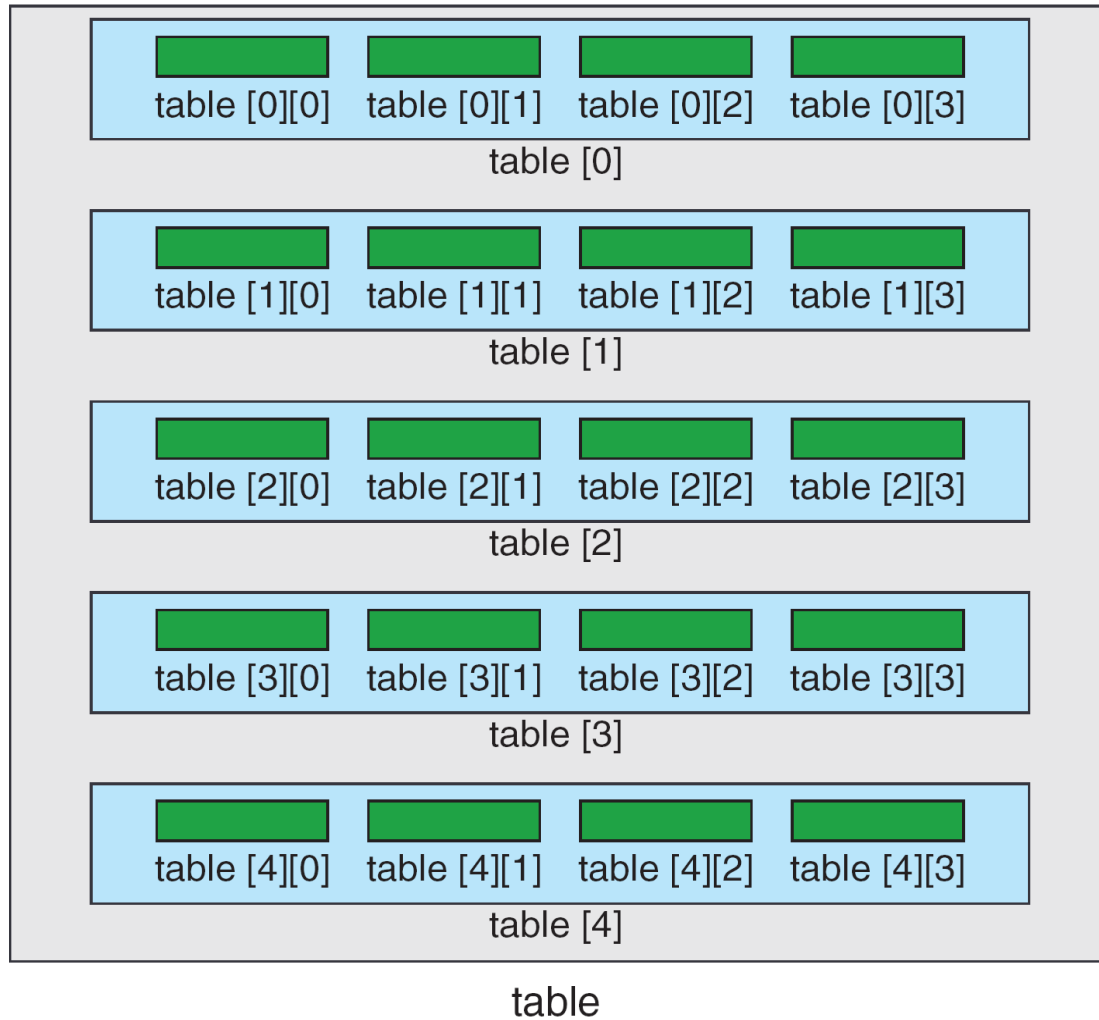


FIGURE C View of Two-dimensional Array

Representation of Two-Dimensional Arrays in Memory

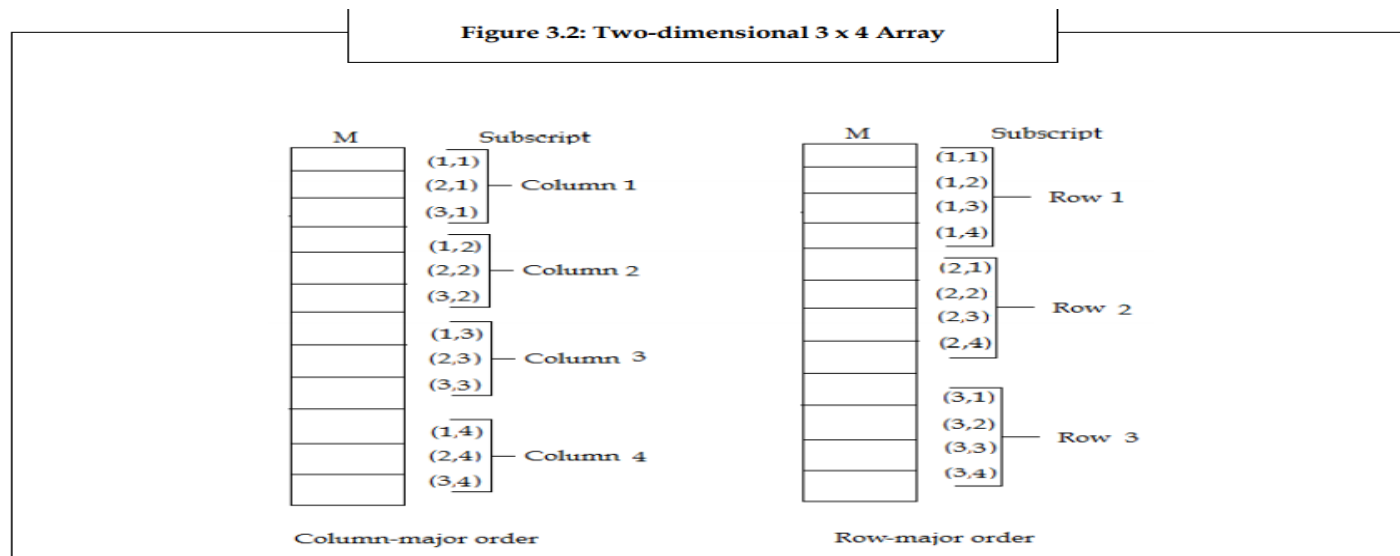
Let A be a two-dimensional $m \times n$ array. Although A is pictured as a rectangular array of elements with m rows and n columns, the array will be represented in memory by a block of $m \cdot n$ sequential memory locations.

Specifically, the programming language will store the array A either

- a) column by column, is what is called **column-major order**, or
- b) row by row, in **row-major order**.

Row-major order is used in C, PL/I; column-major order is used in Fortran, MATLAB.

The following figure shows these two ways when A is a two-dimensional 3×4 array.



Representation of Two-Dimensional Arrays in Memory

To compute the address LOC (A [J, K]) of A [J, K] we can use the following two formulas:

(Column-major order): **LOC (A [J, K]) = Base (A) + w [M (K - 1) + (J - 1)]**

(Row-major order): **LOC (A [J, K]) = Base (A) + w [N (J - 1) + (K - 1)]**

Here Base (A) is the address of the first element A[1,1] of A.

Example: Consider that 25 students are given 4 tests. The students are numbered from 1-25 and the test score is assigned in a 25 x 4 matrix array - MARKS. Suppose Base (MARKS) = 100 and w = 4 bytes, and the program stores two dimensional arrays using row-major order.

The address of MARKS[10,2], that is the marks scored by the tenth student in the second test are as per the formula:

$$\text{LOC (MARKS[J,K])} = \text{Base(MARKS)} + w [N(J-1) + (K-1)]$$

$$\begin{aligned}\text{LOC (MARKS[10,2])} &= 100 + 4 [4(10-1) + (2-1)] \\ &= 100 + 4 [36+1] = 248\end{aligned}$$

- If MARKS is stored using column-major order then what is the address of MARKS[10,2]?

2D Array Implementation in C

Write a program to read the elements of a 2D array and then display the elements of the array.

```
#include<stdio.h>

int main()
{
    int r, c, i, j, m[10][10];
    printf("How many rows and columns: ");
    scanf("%d%d",&r, &c);
    for(i=1 ; i<=r ; i++)
        for(j=1 ; j<=c ; j++)
            scanf("%d",&m[i][j]);
    printf("\nOutput:\n");
    for(i=1 ; i<=r ; i++)
    {
        for(j=1 ; j<=c ; j++)
            printf("%d\t",m[i][j]);
        printf("\n");
    }
    return 0;
}
```

General Multidimensional Arrays

General multidimensional arrays are defined analogously. More specifically, an n -dimensional $m_1 \times m_2 \times \dots \times m_n$ array B is a collection of $m_1 \cdot m_2 \cdot \dots \cdot m_n$ data elements in which each element is specified by a list of n integers—such as $K_1, K_2, \dots, K_n$ —called *subscripts*, with the property that

$$1 \leq K_1 \leq m_1, \quad 1 \leq K_2 \leq m_2, \quad \dots, \quad 1 \leq K_n \leq m_n$$

The element of B with subscripts $K_1, K_2, \dots, K_n$ will be denoted by

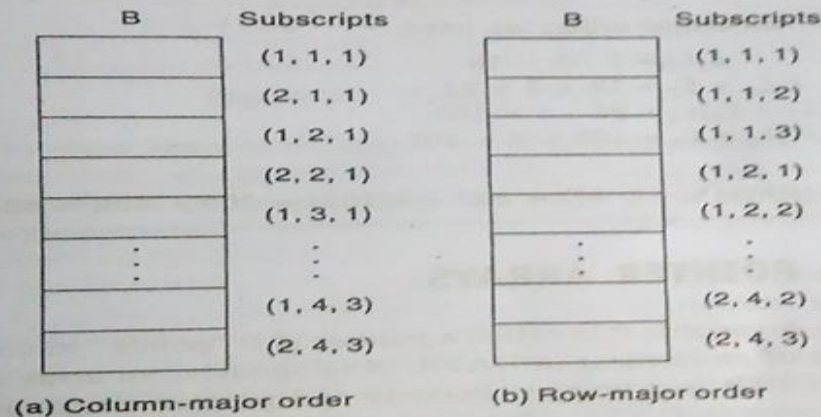
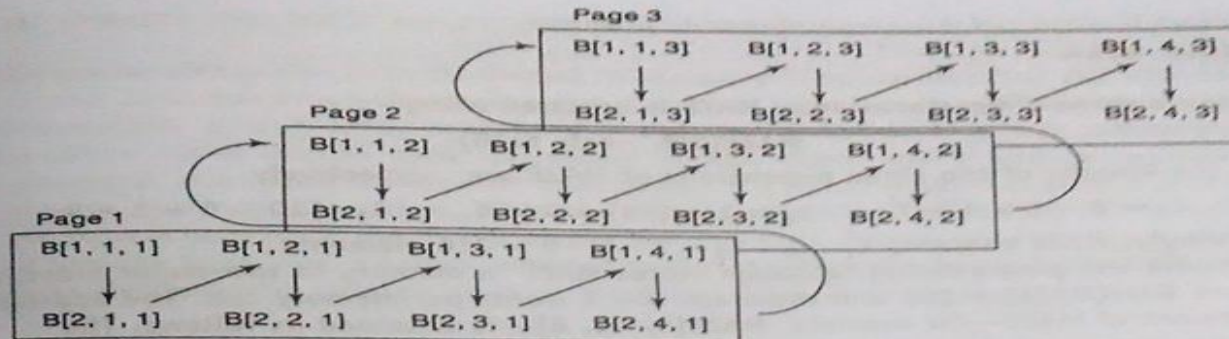
$$B_{K_1, K_2, \dots, K_n} \quad \text{or} \quad B[K_1, K_2, \dots, K_n]$$

The array will be stored in memory in a sequence of memory locations. Specifically, the programming language will store the array B either in row-major order or in column-major order. By *row-major order*, we mean that the elements are listed so that the subscripts vary like an automobile odometer, i.e., so that the last subscript varies first (most rapidly), the next-to-last subscript varies second (less rapidly), and so on. By *column-major order*, we mean that the elements are listed so that the first subscript varies first (most rapidly), the second subscript second (less rapidly), and so on.

Example 4.13

Suppose B is a three-dimensional $2 \times 4 \times 3$ array. Then B contains $2 \cdot 4 \cdot 3 = 24$ elements. These 24 elements of B are usually pictured as in Fig. 4.11; i.e., they appear in three layers, called *pages*, where each page consists of the 2×4 rectangular array of elements with the same third subscript. (Thus the three subscripts of an element in a three-dimensional array are called, respectively, the *row*, *column* and *page* of the element.) The two ways of storing B in memory appear in Fig. 4.12. Observe that the arrows in Fig. 4.11 indicate the column-major order of the elements.

General Multidimensional Arrays



For a given subscript K_i , the effective index E_i of L_i is the number of indices preceding K_i in the index set, and E_i can be calculated from

$$E_i = K_i - \text{lower bound} \quad (4.7)$$

Then the address $\text{LOC}(C[K_1, K_2, \dots, K_N])$ of an arbitrary element of C can be obtained from the formula

$$\text{Base}(C) + w[(((\dots (E_N L_{N-1} + E_{N-1}) L_{N-2}) + \dots + E_3) L_2 + E_2) L_1 + E_1] \quad (4.8)$$

or from the formula

$$\text{Base}(C) + w[(\dots ((E_1 L_2 + E_2) L_3 + E_3) L_4 + \dots + E_{N-1}) L_N + E_N] \quad (4.9)$$

according to whether C is stored in column-major or row-major order. Once again, $\text{Base}(C)$ denotes the address of the first element of C , and w denotes the number of words per memory location.

General Multidimensional Arrays

Example 4.14

Suppose a three-dimensional array MAZE is declared using

$$\text{MAZE}(2:8, -4:1, 6:10)$$

Then the lengths of the three dimensions of MAZE are, respectively,

$$L_1 = 8 - 2 + 1 = 7, \quad L_2 = 1 - (-4) + 1 = 6, \quad L_3 = 10 - 6 + 1 = 5$$

Accordingly, MAZE contains $L_1 \cdot L_2 \cdot L_3 = 7 \cdot 6 \cdot 5 = 210$ elements.

Suppose the programming language stores MAZE in memory in row-major order, and suppose $\text{Base}(\text{MAZE}) = 200$ and there are $w = 4$ words per memory cell. The address of an element of MAZE—for example, $\text{MAZE}[5, -1, 8]$ —is obtained as follows. The effective indices of the subscripts are, respectively,

$$E_1 = 5 - 2 = 3, \quad E_2 = -1 - (-4) = 3, \quad E_3 = 8 - 6 = 2$$

Using Eq. (4.9) for row-major order, we have:

$$E_1 L_2 = 3 \cdot 6 = 18$$

$$E_1 L_2 + E_2 = 18 + 3 = 21$$

$$(E_1 L_2 + E_2) L_3 = 21 \cdot 5 = 105$$

$$(E_1 L_2 + E_2) L_3 + E_3 = 105 + 2 = 107$$

Therefore,

$$\text{LOC}(\text{MAZE}[5, -1, 8]) = 200 + 4(107) = 200 + 428 = 628$$

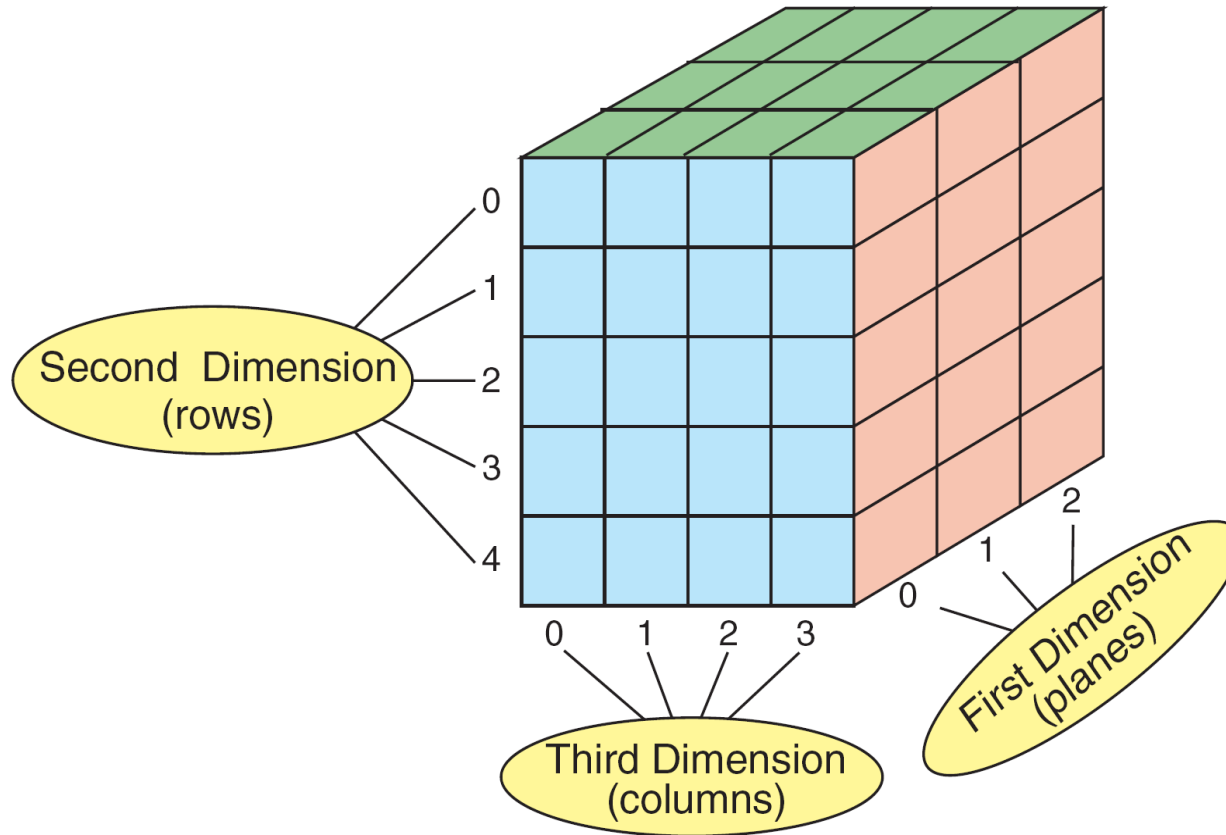


FIGURE 8-40 A Three-dimensional Array (3 x 5 x 4)

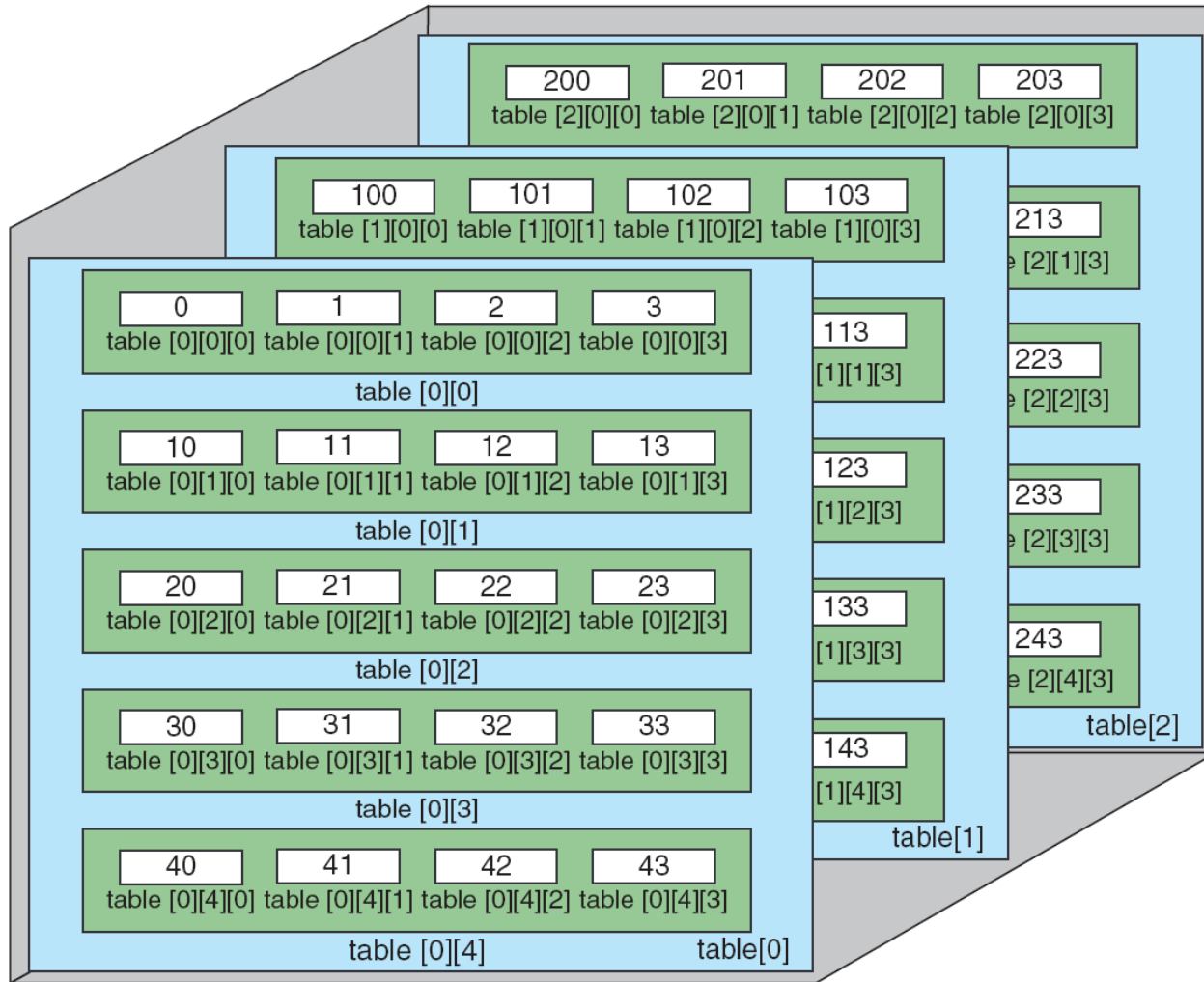


FIGURE 8-41 C View of Three-dimensional Array

Matrices

An $m \times n$ matrix A is an array of $m \cdot n$ numbers arranged in m rows and n columns as follows:

$$A = \begin{pmatrix} A_{11} & A_{12} & A_{13} & \dots & A_{1n} \\ A_{21} & A_{22} & A_{23} & \dots & A_{2n} \\ \dots & \dots & \dots & \dots & \dots \\ A_{m1} & A_{m2} & A_{m3} & \dots & A_{mn} \end{pmatrix}$$

- The **transpose** of a **matrix** is simply a flipped version of the original **matrix**. The result of transposing an $m \times n$ matrix is an $n \times m$ matrix with property:

$$M^T(j,i) = M(i,j), 1 \leq i \leq m, 1 \leq j \leq n.$$

- The sum of matrices is only defined for matrices that have **the same dimensions**. Suppose A and B are $m \times n$ matrices. The *sum* of A and B , written $A + B$, is the $m \times n$ matrix obtained by adding corresponding elements from A and B ;

Suppose

$$A = \begin{pmatrix} 1 & -2 & 3 \\ 0 & 4 & 5 \end{pmatrix} \quad \text{and} \quad B = \begin{pmatrix} 3 & 0 & -6 \\ 2 & -3 & 1 \end{pmatrix}$$

Then:

$$A + B = \begin{pmatrix} 1+3 & -2+0 & 3+(-6) \\ 0+2 & 4+(-3) & 5+1 \end{pmatrix} = \begin{pmatrix} 4 & -2 & -3 \\ 2 & 1 & 6 \end{pmatrix}$$

Matrix Multiplication

The product of matrices A and B is only defined when the **number of columns in A is equal to the number of rows in B** .

Suppose A is an $m \times p$ and suppose B is a $p \times n$ matrix. The product of A and B , written AB , is the $m \times n$ matrix C whose ij th element C_{ij} is given by

$$C_{ij} = A_{i1}B_{1j} + A_{i2}B_{2j} + \dots + A_{ip}B_{pj} = \sum_{k=1}^p A_{ik}B_{kj}$$

(c) Suppose

$$A = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix} \quad \text{and} \quad B = \begin{pmatrix} 2 & 0 & -4 \\ 3 & 2 & 6 \end{pmatrix}$$

The product matrix AB is defined and is a 2×3 matrix. The elements in the first row of AB are obtained, respectively, by multiplying the first row of A by each of the columns of B :

$$\begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix} \begin{pmatrix} 2 & 0 & -4 \\ 3 & 2 & 6 \end{pmatrix} = \begin{pmatrix} 1 \cdot 2 + 3 \cdot 3 & 1 \cdot 0 + 3 \cdot 2 & 1 \cdot (-4) + 3 \cdot 6 \end{pmatrix} = \begin{pmatrix} 11 & 6 & 14 \end{pmatrix}$$

Similarly, the elements in the second row of AB are obtained, respectively, by multiplying the second row of A by each of the columns of B :

$$\begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix} \begin{pmatrix} 2 & 0 & -4 \\ 3 & 2 & 6 \end{pmatrix} = \begin{pmatrix} 11 & 6 & 14 \\ 2 \cdot 2 + 4 \cdot 3 & 2 \cdot 0 + 4 \cdot 2 & 2 \cdot (-4) + 4 \cdot 6 \end{pmatrix} = \begin{pmatrix} 11 & 6 & 14 \\ 16 & 8 & 16 \end{pmatrix}$$

That is,
$$AB = \begin{pmatrix} 11 & 6 & 14 \\ 16 & 8 & 16 \end{pmatrix}$$

Matrix Multiplication

(Matrix Multiplication) MATMUL (A, B, C, M, P, N)

Let A be an $M \times P$ matrix array, and let B be a $P \times N$ matrix array. This algorithm stores the product of A and B in an $M \times N$ matrix array C.

1. Repeat Steps 2 to 4 for $I = 1$ to M :
2. Repeat Steps 3 and 4 for $J = 1$ to N :
3. Set $C[I, J] := 0$
4. Repeat for $K = 1$ to P :
 $C[I, J] := C[I, J] + A[I, K] * B[K, J]$
5. Exit.

Sessional:

1. Write a program to interchange the row and column of a matrix.
2. Write a program to add two matrices.
3. Write a program to calculate the multiplication of two matrices.

Sparse Matrices

Matrices with a relatively **high proportion of zero** entries are called *sparse matrices*. If the number of zeros in a matrix exceeds $(n*m)/2$, where n, m is the dimension of the matrix, then the matrix is called **sparse matrix**.

- Two general types of n -square sparse matrices are there which occur in various applications are mention in figure below(It is sometimes custc

$$\begin{pmatrix} 4 & & & & \\ 3 & -5 & & & \\ 1 & 0 & 6 & & \\ -7 & 8 & -1 & 3 & \\ 5 & -2 & 0 & 2 & -8 \end{pmatrix}$$

(a) Triangular matrix

$$\begin{pmatrix} 5 & -3 & & & & \\ 1 & 4 & 3 & & & \\ & 9 & -3 & 6 & & \\ & & 2 & 4 & -7 & \\ & & & 3 & -1 & 0 \\ & & & & 6 & -5 & 8 \\ & & & & & 3 & -1 \end{pmatrix}$$

(b) Tridiagonal matrix

- Triangular matrix This is the matrix where all the entries above the main diagonal are zero or equivalently where non-zero entries can only occur on or below the main diagonal is called a (lower)Triangular matrix.
- Tridiagonal matrix This is the matrix where non-zero entries can only occur on the diagonal or on elements immediately above or below the diagonal is called a Tridiagonal matrix.
- The natural method of representing matrices in memory as two-dimensional arrays may not be suitable for sparse matrices i.e. one may save space by storing only those entries which may be non-zero

Sparse Matrices

Example 4.25

Suppose we want to place in memory the triangular array A in Fig. 4.22. Clearly it would be wasteful to store those entries above the main diagonal of A , since we know they are all zero; hence we store only the other entries of A in a linear array B as indicated by the arrows. That is, we let

$$B[1] = a_{11}, \quad B[2] = a_{21}, \quad B[3] = a_{22}, \quad B[3] = a_{31}, \quad \dots$$

Observe first that B will contain only

$$1 + 2 + 3 + 4 + \dots + n = \frac{1}{2}n(n + 1)$$

elements, which is about half as many elements as a two-dimensional $n \times n$ array. Since we will require the value of a_{jk} in our programs, we will want the formula that gives us the integer L in terms of J and K where

$$B[L] = a_{jk}$$

Observe that L represents the number of elements in the list up to and including a_{jk} . Now there are

$$1 + 2 + 3 + \dots + (J - 1) = \frac{J(J - 1)}{2}$$

elements in the rows above a_{jk} , and there are K elements in row J up to and including a_{jk} . Accordingly,

$$L = \frac{J(J - 1)}{2} + K$$

yields the index that accesses the value a_{jk} from the linear array B .

Sparse Matrices

Home Task:

4.13 Consider an n -square tridiagonal array A as shown in Fig. 4.24. Note that A has n elements on the diagonal and $n - 1$ elements above and $n - 1$ elements below the diagonal. Hence A contains at most $3n - 2$ nonzero elements. Suppose we want to store A in a linear array B as indicated by the arrows in Fig. 4.24; i.e.,

$$B[1] = a_{11}, \quad B[2] = a_{12}, \quad B[3] = a_{21}, \quad B[4] = a_{22}, \quad \dots$$

Find the formula that will give us L in terms of J and K such that

$$B[L] = A[J, K]$$

(so that one can access the value of $A[J, K]$ from the array B).

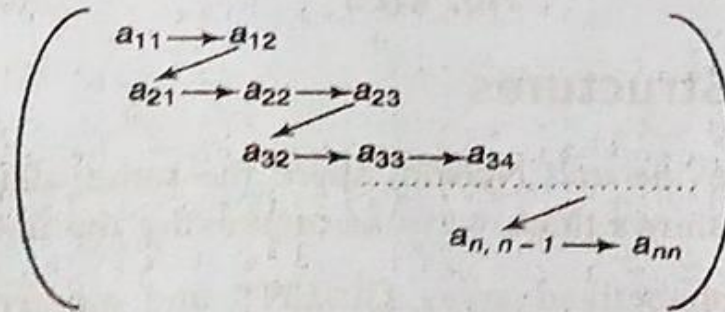


Fig. 4.24 Tridiagonal Array

International Islamic University Chittagong

Department of Computer Science & Engineering

Course Code: CSE-2321

Course Title: Data Structures

Prepared By

Mohammed Shamsul Alam
Professor,
Dept. of CSE, IIUC

Presented By

Mysha Sarin Kabisha
Assistant Lecturer,
Dept. of CSE, IIUC

Lecture – 6

String

String

- A finite sequence S of zero or more characters is called a **String**.
- The string with zero character is called the **empty string** or **null string**.
- The number of characters in a string is called its **length**.
- Specific string will be denoted by enclosing their character in **single quotation mark**. For example:

‘THE END’ ‘TO BE OR NOT TO BE’ “ ‘123’
are strings with lengths 7, 18, 0 and 3.

- Let S_1 and S_2 be the strings. The string consisting of the character of S_1 followed by the characters of S_2 is called the **concatenation** of S_1 and S_2 . It will be denoted by $S_1 \parallel S_2$. For example,

$S_1 = \text{'THE'}$ $S_2 = \text{'END'}$ then

$S_1 \parallel S_2 = \text{'THEEND'}$

$S_1 \parallel \text{' ' } \parallel S_2 = \text{'THE END'}$ Here ‘ ’ means blank space

- The length of $S_1 \parallel S_2$ is equal to the sum of lengths of the strings S_1 and S_2 .

String

- A string Y is called a **substring** of a string S if there exists strings X and Z such that $S = X \parallel Y \parallel Z$.
- If X is an empty string, then Y is called *initial substring* of S, and if Z is an empty string then Y is called *terminal substring* of S.
- For example
 - 'BE OR NOT' is a substring of 'TO BE OR NOT TO BE'
 - 'THE' is an initial substring of 'THE END'
 - 'END' is a terminal substring of 'THE END'
- If Y is a substring of S then the length of Y cannot exceed the length of S.

String Operations

Length

The number of characters in a string is called its **length**. We will write

LENGTH (string)

for the length of a given string. For example,

LENGTH ('Computer') = 8

LENGTH ('0') = 1

Concatenation

Let S1 and S2 be strings. The concatenation of S1 and S2 is denoted by **S1 || S2** , is the string consisting of the characters of S1 followed by the characters of S2 . For example,

Suppose S1 = 'Kazi' and S2 = 'Nazrul' then

S1 || S2 = 'KaziNazrul'

S1 || ' ' || S2 = 'Kazi Nazrul'

String Operations

Substring

Accessing a substring from a given string requires 3 pieces of information.

- i) The name of the string or the string itself
- ii) The position of the first character of the substring in the given string
- iii) The length of the substring or position of the last character of the substring

We call this operation SUBSTRING. Specifically, we write

SUBSTRING (String, initial, length)

to denote the substring of a string S beginning in a position K and having a length L or **SUBSTRING (S, K, L)**.

For example,

SUBSTRING ('TO BE OR NOT TO BE', 4, 7) = 'BE OR N'

SUBSTRING ('THE END', 4, 4) = '□END'

String Operations

Indexing

Indexing also called *pattern matching*, refers to finding the position where a string pattern **P** first appears in a given string text **T**.

We call this operation INDEX and write as

INDEX (text, pattern)

- If the pattern **P** does not appear in text **T** then INDEX is assigned the value 0.
- The arguments text and pattern can be either string constants or string variables.

For example,

T = 'HIS FATHER IS THE PROFESSOR'

Then

INDEX (T, 'THE') = 7

INDEX (T, 'THEN') = 0

INDEX (T, ' THE') = 14

Word Processing

Insertion

Suppose in a given text T we want to insert a string S so that S begins in position K . We denote this operation by

$$\text{INSERT}(\text{text}, \text{position}, \text{string})$$

For example,

$$\text{INSERT}('ABCDEFGH', 3, 'XYZ') = 'ABXYZCDEFGH'$$
$$\text{INSERT}('ABCDEFGH', 6, 'XYZ') = 'ABCDEXYZFGH'$$

Deletion

Suppose in a given text T we want to delete the substring which begins in position K and has length L . We denote this operation by

$$\text{DELETE}(\text{text}, \text{position}, \text{length})$$

For example,

$$\text{DELETE}('ABCDEFGH', 4, 2) = 'ABCDFG'$$
$$\text{DELETE}('ABCDEFGH', 2, 4) = 'AFG'$$

We assume that nothing is deleted if position $K = 0$. Thus

$$\text{DELETE}('ABCDEFGH', 0, 2) = 'ABCDEFGH'$$

Word Processing

- (a) Suppose Algorithm 3.1 is run with the data

$$T = XABYABZ, \quad P = AB$$

Then the loop in the algorithm will be executed twice. During the first execution, the first occurrence of AB in T is deleted, with the result that $T = XYABZ$. During the second execution, the remaining occurrence of AB in T is deleted, so that $T = XYZ$. Accordingly, XYZ is the output.

- (b) Suppose Algorithm 3.1 is run with the data

$$T = XAAABBBY, \quad P = AB$$

Observe that the pattern AB occurs only once in T but the loop in the algorithm will be executed three times. Specifically, after AB is deleted the first time from T we have $T = XAABBY$, and hence AB appears again in T. After AB is deleted a second time from T, we see that $T = XABY$ and AB still occurs in T. Finally, after AB is deleted a third time from T, we have $T = XY$ and AB does not appear in T, and thus $\text{INDEX}(T, P) = 0$. Hence XY is the output.

Word Processing

Replacement

Suppose in a given text T we want to replace the first occurrence of a pattern P_1 by a pattern P_2 . We will denote this operation by

$\text{REPLACE}(\text{text}, \text{pattern}_1, \text{pattern}_2)$

For example

$\text{REPLACE}(\text{'XABYABZ'}, \text{'AB'}, \text{'C'}) = \text{'XCYABZ'}$

$\text{REPLACE}(\text{'XABYABZ'}, \text{'BA'}, \text{'C'}) = \text{'XABYABZ'}$

In the second case, the pattern BA does not occur, and hence there is no change.

Replacement [Every occurrence of the pattern P in T by Q]

$T = \text{XABYABZ}, \quad P = \text{AB}, \quad Q = \text{C}$

Then the loop in the algorithm will be executed twice. During the first execution, the first occurrence of AB in T is replaced by C to yield $T = \text{XCYABZ}$. During the second execution, the remaining AB in T is replaced by C to yield $T = \text{XCYZCZ}$. Hence XCYZCZ is the output.

Word Processing

Replacement [Every occurrence of the pattern P in T by Q]

(b) Suppose Algorithm 3.2 is run with the data

$$T = XAY, \quad P = A, \quad Q = AB$$

Then the algorithm will never terminate. The reason for this is that P will always occur in the text T, no matter how many times the loop is executed. Specifically,

$T = XABY$ at the end of the first execution of the loop

$T = XAB^2Y$ at the end of the second execution of the loop

.....

$T = XAB^nY$ at the end of the n th execution of the loop

(The infinite loop arises here since P is a substring of Q.)

NB:

suppose the length of Q is smaller than the length of P. Then the length of T after each replacement decreases. This guarantees that in this special case where Q is smaller than P the algorithm must terminate.

Pattern Matching Algorithm

Pattern matching is the problem of deciding whether or not a given string pattern P appears in a string text T . We assume that the length of P does not exceed the length of T . This section discusses two pattern matching algorithms. We also discuss the complexity of the algorithms so we can compare their efficiencies.

Remark: During the discussion of pattern matching algorithms, characters are sometimes denoted by lowercase letters ($a, b, c, \dots$) and exponents may be used to denote repetition; e.g.,

$$a^2b^3ab^2 \text{ for } aabbbabb \quad \text{and} \quad (cd)^3 \text{ for } cdc dcd$$

In addition, the empty string may be denoted by Λ , the Greek letter lambda, and the concatenation of strings X and Y may be denoted by $X \cdot Y$ or, simply, XY .

Pattern Matching Algorithm

(Pattern Matching) P and T are strings with lengths R and S, respectively, and are stored as arrays with one character per element. This algorithm finds the INDEX of P in T.

1. [Initialize.] Set $K := 1$ and $MAX := S - R + 1$.
2. Repeat Steps 3 to 5 while $K \leq MAX$:
3. Repeat for $L := 1$ to R : [Tests each character of P.]
 If $P[L] \neq T[K + L - 1]$, then: Go to Step 5.
 [End of inner loop.]
4. [Success.] Set $INDEX = K$, and Exit.
5. Set $K := K + 1$.
 [End of Step 2 outer loop.]
6. [Failure.] Set $INDEX = 0$.
7. Exit.

Activate Windows

Pattern Matching Algorithm

Pattern: **abaa**; searched string: **ababbaabaaab**

ababbaabaaab

.....

abaa_____	step 1	ABA#	mismatch: 4th letter
_abaa_____	step 2	_#...	mismatch: 1st letter
__abaa_____	step 3	__AB#.	mismatch: 3rd letter
___abaa_____	step 4	___#...	mismatch: 1st letter
____abaa_____	step 5	____#...	mismatch: 1st letter
_____abaa__	step 6	_____A#..	mismatch: 2nd letter
_____abaa__	step 7	_____ABAA	success

Pattern Matching Algorithm

```
for ( j = 0; j <= n - m; j++ ) {  
    for ( i = 0; i < m && x[i] == y[i + j]; i++ );  
    if ( i >= m ) return j;  
}
```

Main features of this easy (but slow) $O(nm)$ algorithm:

- No preprocessing phase
- Only constant extra space needed
- Always shifts the window by exactly 1 position to the right
- Comparisons can be done in any order
- mn expected text characters comparisons

Pattern Matching Algorithm

Single Pattern Algorithms (Summary)

Notation:

m – the length (size) of the pattern; n – the length of the searched text

String search algorithm	Time complexity for	
	preprocessing	matching
Naïve	0 (none)	$\Theta(n \cdot m)$
Rabin-Karp	$\Theta(m)$	avg $\Theta(n + m)$ worst $\Theta(n \cdot m)$
Finite state automaton	$\Theta(m \Sigma)$	$\Theta(n)$
Knuth-Morris-Pratt	$\Theta(m)$	$\Theta(n)$
Boyer-Moore	$\Theta(m + \Sigma)$	$\Omega(n/m), O(n)$
Bit based (approximate)	$\Theta(m + \Sigma)$	$\Theta(n)$

See <http://www-igm.univ-mlv.fr/~lecroq/string> for some animations of these and many other string algorithms

International Islamic University Chittagong

Department of Computer Science & Engineering

Course Code: CSE-2321

Course Title: Data Structures

Prepared By

Mohammed Shamsul Alam
Professor,
Dept. of CSE, IIUC

Presented By

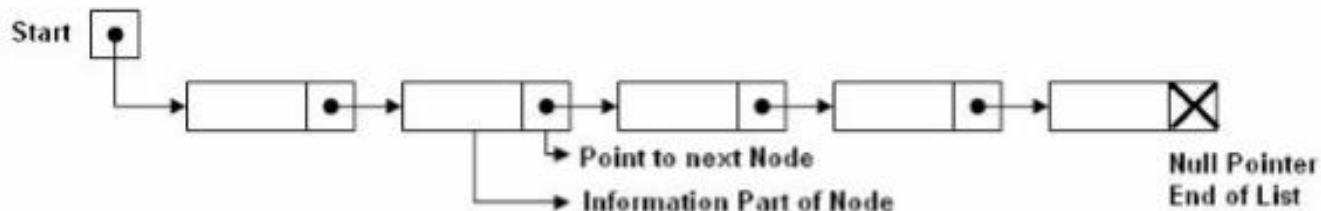
Mysha Sarin Kabisha
Assistant Lecturer,
Dept. of CSE, IIUC

Lecture – 7

Linked List

Linked list

- A **linked list**, or **one-way list**, is a linear collection of data elements, called **nodes**, where the linear order is maintained by means of **links** or **pointers**.
- In linked list, each node is divided into two parts: the first part contains the information of the element, and the second part, called the *link field* or *next pointer field*, contains the address of the next node in the list.
- The pointer of the **last node** contains a special value, called **null** pointer, which is any invalid address. The null pointer, denoted by x in the diagram, signals the end of the list.
- There is a special pointer **START** - which contains the address of **first node** in the list.
- **START = NULL** means list has no nodes. Such a list is called the *null list* or *empty list*.
- The following Fig shows a schematic diagram of a linked list with five nodes.



Linked list

- Linked lists were developed in the year **1955-56** by **Allen Newell, Cliff Shaw, and Herbert Simon** at **RAND Corporation** as a primary data structure. It was developed for their Information Processing Language (IPL). IPL was used by the authors to develop several early artificial intelligent programs, including Logic Theory Machine, the General Problem Solver, and a computer chess program.

The drawbacks of using arrays are:

1. Once the elements are stored, it becomes difficult to insert or delete an element at any position in a list.
2. Arrays have fixed size. Hence, if the memory allocated is too large than the actual data, unused portion of the memory is wasted. If the allocated memory is less, it will result in loss of data due to inadequate memory.

The advantage of using linked list is its ability to dynamically shrink and expand in size. This allows you to insert or delete elements efficiently at any position in the list.

Representation of the linked list in memory

Let LIST be a linked list. Then LIST will be maintained in memory, unless otherwise specified or implied, as follows. First of all, LIST requires two linear arrays—we will call them here INFO and LINK—such that INFO[K] and LINK[K] contain, respectively, the information part and the nextpointer field of a node of LIST. As noted above, LIST also requires a variable name—such as START—which contains the location of the beginning of the list, and a nextpointer sentinel—denoted by NULL—which indicates the end of the list. Since the subscripts of the arrays INFO and LINK will usually be positive, we will choose NULL = 0, unless otherwise stated.

Example 5.2

Figure 5.4 pictures a linked list in memory where each node of the list contains a single character. We can obtain the actual list of characters, or, in other words, the string, as follows:

START = 9, so INFO[9] = N is the first character.
LINK[9] = 3, so INFO[3] = O is the second character.
LINK[3] = 6, so INFO[6] = □ (blank) is the third character.
LINK[6] = 11, so INFO[11] = E is the fourth character.
LINK[11] = 7, so INFO[7] = X is the fifth character.
LINK[7] = 10, so INFO[10] = I is the sixth character.
LINK[10] = 4, so INFO[4] = T is the seventh character.
LINK[4] = 0, the NULL value, so the list has ended.

In other words, NO EXIT is the character string.

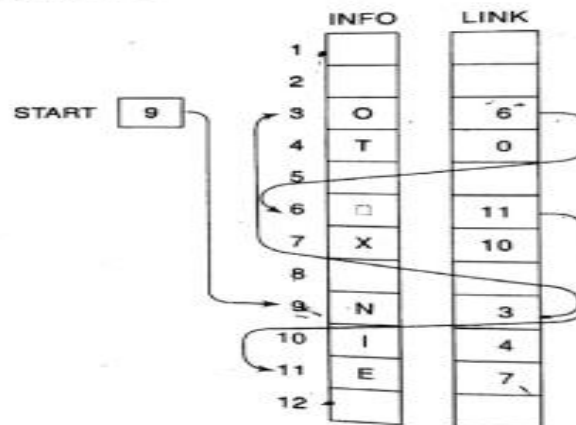
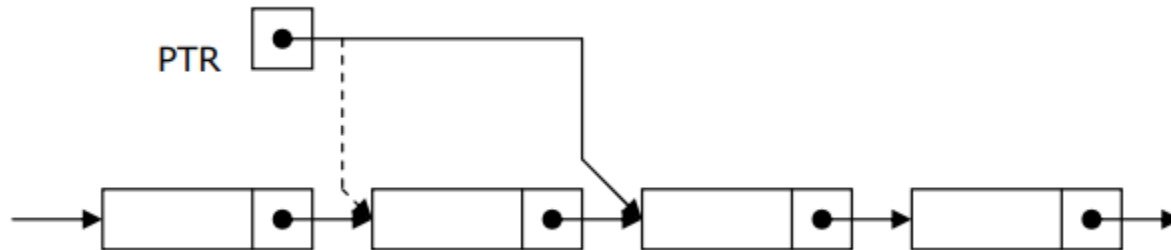


Fig. 5.4

Traversing a Linked List

Algorithm: (Traversing a Linked List) Let LIST be a linked list in memory. This algorithm traverses LIST, applying an operation PROCESS to each element of list. The variable PTR point to the node currently being processed.

1. Set PTR=START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR!=NULL.
 3. Apply PROCESS to INFO[PTR]. // PTR-> INFO
 4. Set PTR = LINK [PTR] [PTR now points to the next node.]
 // PTR= PTR-> NEXT
 [End of Step 2 loop.]
5. Exit.



Traversing a Linked List

Example 5.6

The following procedure prints the information at each node of a linked list. Since the procedure must traverse the list, it will be very similar to Algorithm 5.1.

Procedure: PRINT(INFO, LINK, START)

This procedure prints the information at each node of the list.

1. Set PTR := START.
2. Repeat Steps 3 and 4 while PTR $\neq$ NULL:
3. Write: INFO[PTR].
4. Set PTR := LINK[PTR]. [Updates pointer.]
- [End of Step 2 loop.]
5. Return.

Example 5.7

The following procedure finds the number NUM of elements in a linked list.

Procedure: COUNT(INFO, LINK, START, NUM)

1. Set NUM := 0. [Initializes counter.]
2. Set PTR := START. [Initializes pointer.]
3. Repeat Steps 4 and 5 while PTR $\neq$ NULL.
4. Set NUM := NUM + 1. [Increases NUM by 1.]
5. Set PTR := LINK[PTR]. [Updates pointer.]
- [End of Step 3 loop.]
6. Return.

Searching a Linked List

Algorithm: SEARCH(INFO, NEXT, HEAD, ITEM, PREV, CURR, SCAN)
LIST is a linked list in the memory. This algorithm finds the location LOC of the node where ITEM first appear in LIST, otherwise sets LOC=NULL.

1. Set PTR=START.
2. Repeat Step 3 and 4 while PTR≠NULL:
3. if ITEM = INFO [PTR] then:

// ITEM = PTR->INFO

Set LOC=PTR, and Exit.

[Search is successful.]

Else:

Set PTR=LNK [PTR]

// PTR = PTR->NEXT

[End of Step 2 loop.]
4. Set LOC=NULL

[Search is unsuccessful.]
5. Exit.

Searching a Sorted Linked List

Suppose the data in LIST are sorted. Again we search for ITEM in LIST by traversing the list using a pointer variable PTR and comparing ITEM with the contents INFO[PTR] of each node, one by one, of LIST. Now, however, we can stop once ITEM exceeds INFO[PTR]. The algorithm follows.

Algorithm 5.3: SRCHSL(INFO, LINK, START, ITEM, LOC)

LIST is a sorted list in memory. This algorithm finds the location LOC of the node where ITEM first appears in LIST, or sets $LOC \doteq NULL$.

1. Set $PTR := START$.
2. Repeat Step 3 while $PTR \neq NULL$:
3. If $ITEM < INFO[PTR]$, then:
 Set $PTR := LINK[PTR]$. [PTR now points to next node.]
 Else if $ITEM = INFO[PTR]$, then:
 Set $LOC := PTR$, and Exit. [Search is successful.]
 Else:
 Set $LOC := NULL$, and Exit. [ITEM now exceeds INFO[PTR].]
 [End of If structure.]
 [End of Step 2 loop.]
4. Set $LOC := NULL$.
5. Exit.

The complexity of this algorithm is still the same as that of other linear search algorithms; that is, the worst-case running time is proportional to the number n of elements in LIST, and the average-case running time is approximately proportional to $n/2$.

- Binary search algorithm can not be applied to a sorted linked list, since there is no way of indexing the middle element in the list.

Array vs Linked list

	Array	Linked List
Strength	• Random Access (Fast Search Time) • Less memory needed per element • Better cache locality	• Fast Insertion/Deletion Time • Dynamic Size • Efficient memory allocation/utilization
Weakness	• Slow Insertion/Deletion Time • Fixed Size • Inefficient memory allocation/utilization	• Slow Search Time • More memory needed per node as additional storage required for pointers

International Islamic University Chittagong

Department of Computer Science & Engineering

Course Code: CSE-2321

Course Title: Data Structures

Prepared By

Mohammed Shamsul Alam
Professor,
Dept. of CSE, IIUC

Presented By

Mysha Sarin Kabisha
Assistant Lecturer,
Dept. of CSE, IIUC

Lecture – 8

Linked List-2

Avail list

- Together with the linked lists in memory. A special list is maintained which consists of **unused memory cells**. This list, which has its own pointer, is called **Avail list** or the **free storage list** or the **free pool**.

Suppose the list of patients in Example 5.1 is stored in the linear arrays BED and LINK (so that the patient in bed K is assigned to BED[K]). Then the available space in the linear array BED may be linked as in Fig. 5.9. Observe that BED[10] is the first available bed, BED[2] is the next available bed, and BED[6] is the last available bed. Hence BED[6] has the null pointer in its nextpointer field; that is, LINK[6] = 0.

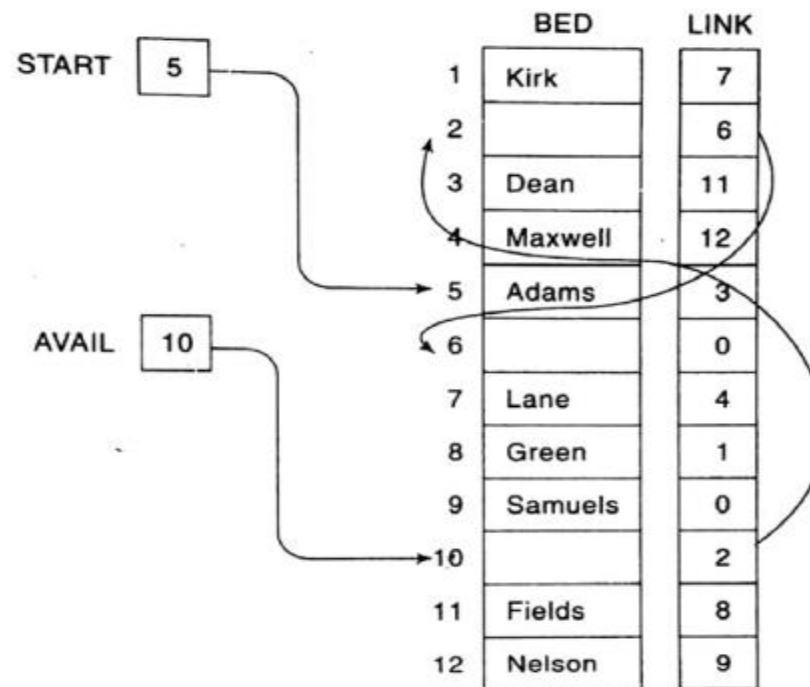


Fig. 5.9

Garbage collection

- The operating system of a computer may periodically collect all the deleted space on to the free storage list. Any technique which does these collections is called **garbage collection**.
- The operating system will perform this operation whenever it finds the CPU idle or whenever the programs are falling short of memory space.

Overflow and Underflow

Sometimes new data are to be inserted into a data structure but there is no available space, i.e., the free-storage list is empty. This situation is usually called *overflow*. The programmer may handle overflow by printing the message OVERFLOW. In such a case, the programmer may then modify the program by adding space to the underlying arrays. Observe that overflow will occur with our linked lists when AVAIL = NULL and there is an insertion.

Analogously, the term *underflow* refers to the situation where one wants to delete data from a data structure that is empty. The programmer may handle underflow by printing the message UNDERFLOW. Observe that underflow will occur with our linked lists when START = NULL and there is a deletion.

Insertion into a Linked List

Let LIST be a linked list with successive nodes A and B, as pictured in Fig. 5.14(a). Suppose a node N is to be inserted into the list between nodes A and B. The schematic diagram of such an insertion appears in Fig. 5.14(b). That is, node A now points to the new node N, and node N points to node B, to which A previously pointed.

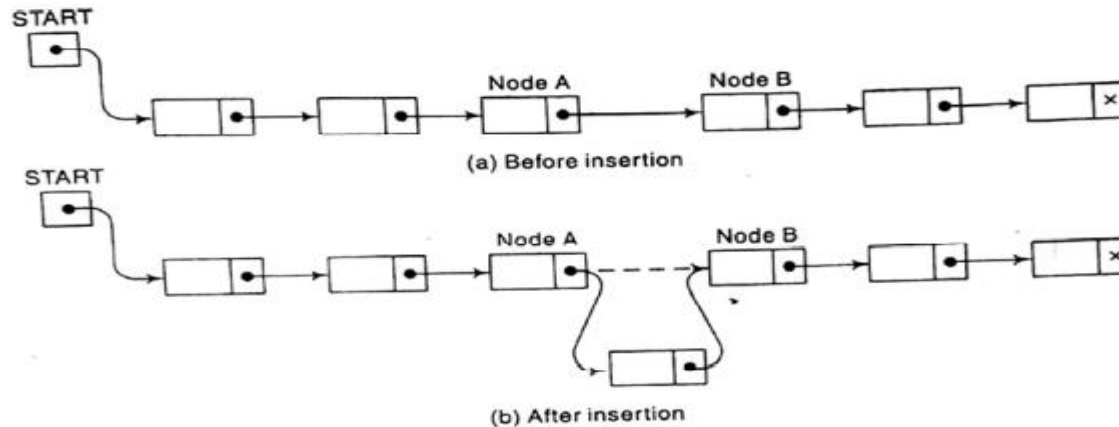


Fig. 5.14

Figure 5.14 does not take into account that the memory space for the new node N will come from the AVAIL list. Specifically, for easier processing, the first node in the AVAIL list will be used for the new node N. Thus a more exact schematic diagram of such an insertion is that in Fig. 5.15. Observe that three pointer fields are changed as follows:

- (1) The nextpointer field of node A now points to the new node N, to which AVAIL previously pointed.

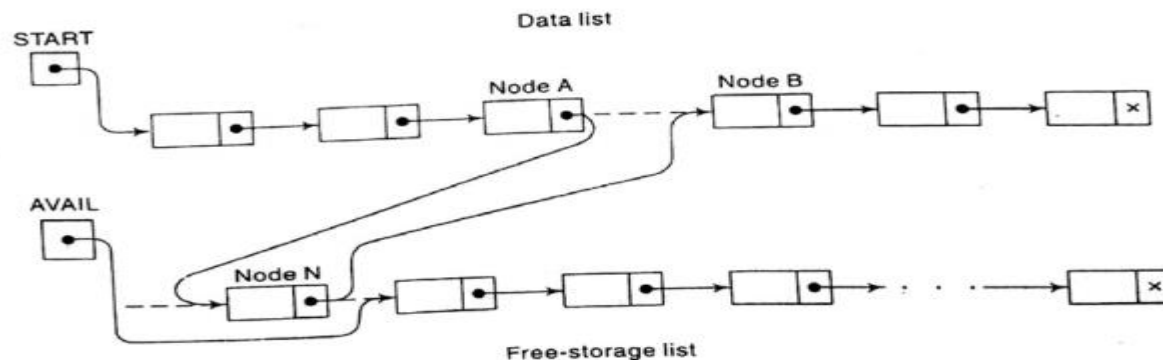


Fig. 5.15

Inserting at the beginning of a Linked List

- (2) AVAIL now points to the second node in the free pool, to which node N previously pointed
- (3) The nextpointer field of node N now points to node B, to which node A previously pointed

There are also two special cases. If the new node N is the first node in the list, then START will point to N; and if the new node N is the last node in the list, then N will contain the null pointer

Algorithm 5.4: INSFIRST(INFO, LINK, START, AVAIL, ITEM)

This algorithm inserts ITEM as the first node in the list.

1. [OVERFLOW?] If AVAIL = NULL, then: Write: OVERFLOW, and Exit.
2. [Remove first node from AVAIL list.]
Set NEW := AVAIL and AVAIL := LINK[AVAIL].
3. Set INFO[NEW] := ITEM. [Copies new data into new node]
4. Set LINK[NEW] := START. [New node now points to original first node.]
5. Set START := NEW. [Changes START so it points to the new node.]
6. Exit.

Steps 1 to 3 have already been discussed, and the schematic diagram of Steps 2 and 3 appears in Fig. 5.17. The schematic diagram of Steps 4 and 5 appears in Fig. 5.18.

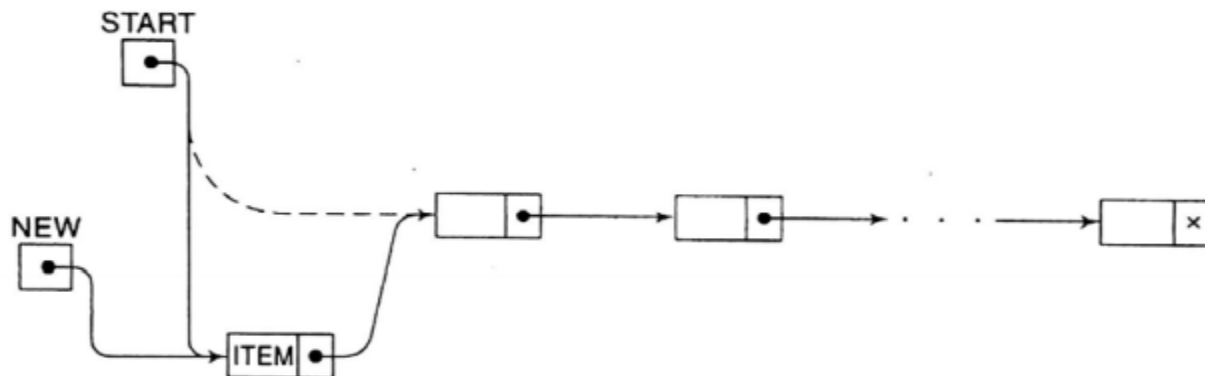


Fig. 5.18 Insertion at the Beginning of a List

Inserting after a given node

Algorithm 5.5: INSLOC(INFO, LINK, START, AVAIL, LOC, ITEM)
This algorithm inserts ITEM so that ITEM follows the node with location LOC or inserts ITEM as the first node when LOC = NULL.

1. [OVERFLOW?] If AVAIL = NULL, then: Write: OVERFLOW, and Exit.
2. [Remove first node from AVAIL list.]
Set NEW := AVAIL and AVAIL := LINK[AVAIL].
3. Set INFO[NEW] := ITEM. [Copies new data into new node.]
4. If LOC = NULL, then: [Insert as first node.]
Set LINK[NEW] := START and START := NEW.
Else: [Insert after node with location LOC.]
Set LINK [NEW] := LINK[LOC] and LINK[LOC] := NEW.
[End of If structure.]
5. Exit.

Inserting into a sorted Linked list

Procedure 5.6: FINDA(INFO, LINK, START, ITEM, LOC)

This procedure finds the location LOC of the last node in a sorted list such that $\text{INFO}[\text{LOC}] < \text{ITEM}$, or sets $\text{LOC} = \text{NULL}$.

1. [List empty?] If $\text{START} = \text{NULL}$, then: Set $\text{LOC} := \text{NULL}$, and Return.
2. [Special case?] If $\text{ITEM} < \text{INFO}[\text{START}]$, then: Set $\text{LOC} := \text{NULL}$, and Return.
3. Set $\text{SAVE} := \text{START}$ and $\text{PTR} := \text{LINK}[\text{START}]$. [Initializes pointers.]

4. Repeat Steps 5 and 6 while $\text{PTR} \neq \text{NULL}$.

5. If $\text{ITEM} < \text{INFO}[\text{PTR}]$, then:
Set $\text{LOC} := \text{SAVE}$, and Return.

[End of If structure.]

6. Set $\text{SAVE} := \text{PTR}$ and $\text{PTR} := \text{LINK}[\text{PTR}]$. [Updates pointers.]

[End of Step 4 loop.]

7. Set $\text{LOC} := \text{SAVE}$.

8. Return.

Now we have all the components to present an algorithm which inserts ITEM into a linked list. The simplicity of the algorithm comes from using the previous two procedures.

Algorithm 5.7: INSERT(INFO, LINK, START, AVAIL, ITEM)

This algorithm inserts ITEM into a sorted linked list.

1. [Use Procedure 5.6 to find the location of the node preceding ITEM.]
Call $\text{FINDA}(\text{INFO}, \text{LINK}, \text{START}, \text{ITEM}, \text{LOC})$.
2. [Use Algorithm 5.5 to insert ITEM after the node with location LOC.]
Call $\text{INSLOC}(\text{INFO}, \text{LINK}, \text{START}, \text{AVAIL}, \text{LOC}, \text{ITEM})$.
3. Exit.

Inserting into a sorted Linked list

Figure 5.21 shows the data structure after Jones is added to the patient list. We emphasize that only three pointers have been changed, AVAIL, LINK[10] and LINK[8].

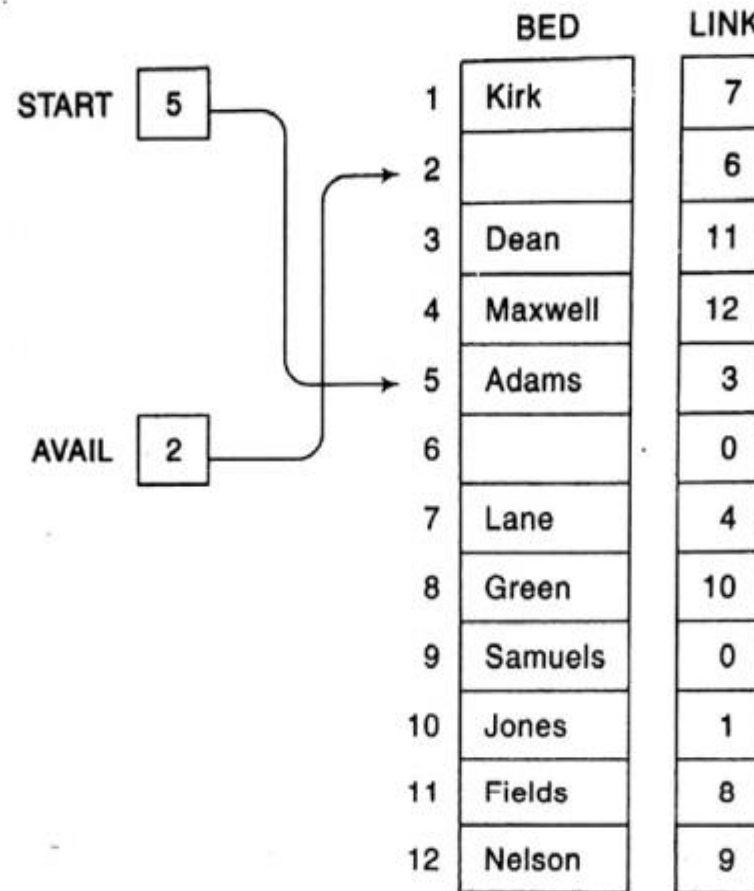


Fig. 5.21

Deletion from a Linked List

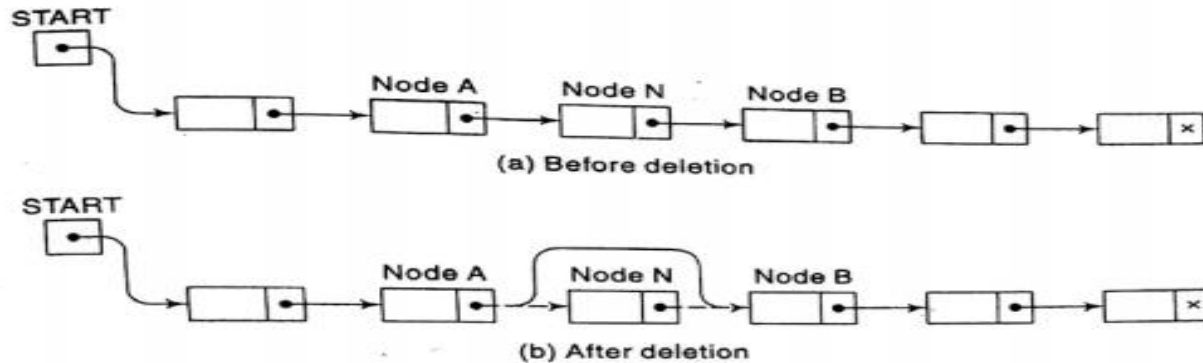


Fig. 5.22

Figure 5.22 does not take into account the fact that, when a node N is deleted from our list, we will immediately return its memory space to the AVAIL list. Specifically, for easier processing, it will be returned to the beginning of the AVAIL list. Thus a more exact schematic diagram of such a deletion is the one in Fig. 5.23. Observe that three pointer fields are changed as follows:

- (1) The nextpointer field of node A now points to node B, where node N previously pointed.
- (2) The nextpointer field of N now points to the original first node in the free pool, where AVAIL previously pointed.
- (3) AVAIL now points to the deleted node N.

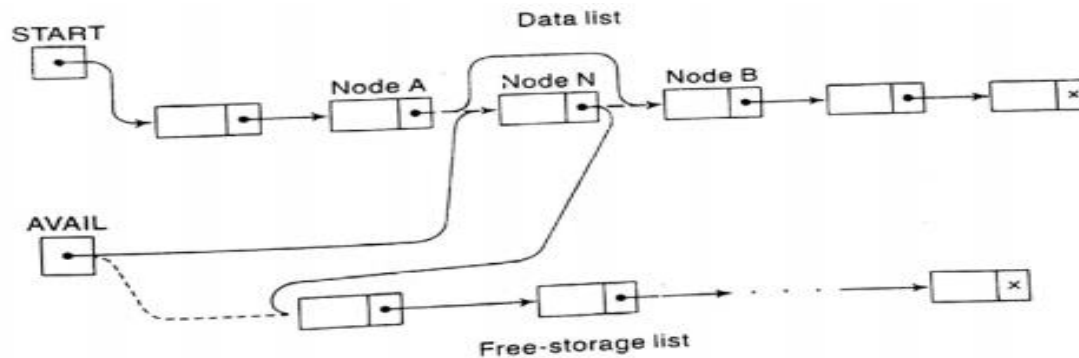


Fig. 5.23

There are also two special cases. If the deleted node N is the first node in the list, then START will point to node B; and if the deleted node N is the last node in the list, then node A will contain the NULL pointer.

Deleting the node following a given node

Algorithm 5.8: DEL(INFO, LINK, START, AVAIL, LOC, LOCP)
This algorithm deletes the node N with location LOC. LOCP is the location of the node which precedes N or, when N is the first node, LOCP = NULL.

1. If LOCP = NULL, then:

Set START := LINK[START]. [Deletes first node.]

Else:

Set LINK[LOCP] := LINK[LOC]. [Deletes node N.]

[End of If structure.]

2. [Return deleted node to the AVAIL list.]

Set LINK[LOC] := AVAIL and AVAIL := LOC.

3. Exit.

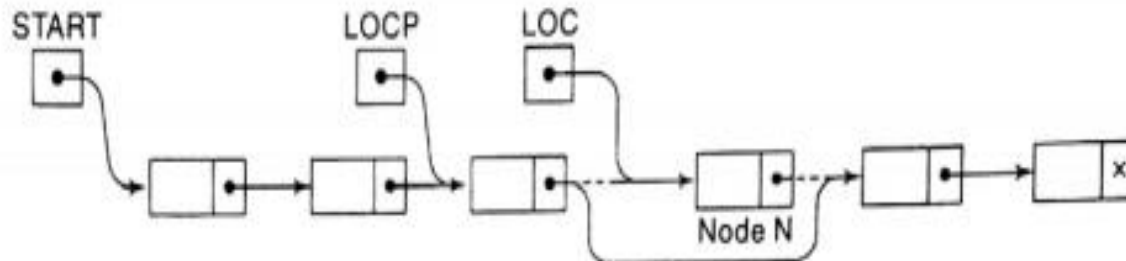


Fig. 5.27 $LINK[LOCP] := LINK[LOC]$

Deleting the node with a given ITEM

Procedure 5.9: FINDB(INFO, LINK, START, ITEM, LOC, LOCP)

This procedure finds the location LOC of the first node N which contains ITEM and the location LOCP of the node preceding N. If ITEM does not appear in the list, then the procedure sets LOC = NULL; and if ITEM appears in the first node, then it sets LOCP = NULL.

1. [List empty?] If START = NULL, then:
 Set LOC := NULL and LOCP := NULL, and Return.
 [End of If structure.]
2. [ITEM in first node?] If INFO[START] = ITEM, then:
 Set LOC := START and LOCP = NULL, and Return.
 [End of If structure.]
3. Set SAVE := START and PTR := LINK[START]. [Initializes pointers.]
4. Repeat Steps 5 and 6 while PTR ≠ NULL.
5. If INFO[PTR] = ITEM, then:
 Set LOC := PTR and LOCP := SAVE, and Return.
 [End of If structure.]
6. Set SAVE := PTR and PTR := LINK[PTR]. [Updates pointers.]
 [End of Step 4 loop.]
7. Set LOC := NULL. [Search unsuccessful.]
8. Return.

Algorithm 5.10: DELETE(INFO, LINK, START, AVAIL, ITEM)

This algorithm deletes from a linked list the first node N which contains the given ITEM of information.

1. [Use Procedure 5.9 to find the location of N and its preceding node.]
 Call FINDB(INFO, LINK, START, ITEM, LOC, LOCP)
2. If LOC = NULL, then: Write: ITEM not in list, and Exit.
3. [Delete node.]
 If LOCP = NULL, then:
 Set START := LINK[START]. [Deletes first node.]
 Else:
 Set LINK[LOCP] := LINK[LOC].
 [End of If structure.]
4. [Return deleted node to the AVAIL list.]
 Set LINK[LOC] := AVAIL and AVAIL := LOC.
5. Exit.

Deleting the node with a given ITEM

Consider the list of patients in Fig. 5.21. Suppose the patient Green is discharged. We simulate Procedure 5.9 to find the location LOC of Green and the location LOCP of the patient preceding Green. Then we simulate Algorithm 5.10 to delete Green from the list. Here ITEM = Green, INFO = BED, START = 5 and AVAIL = 2.

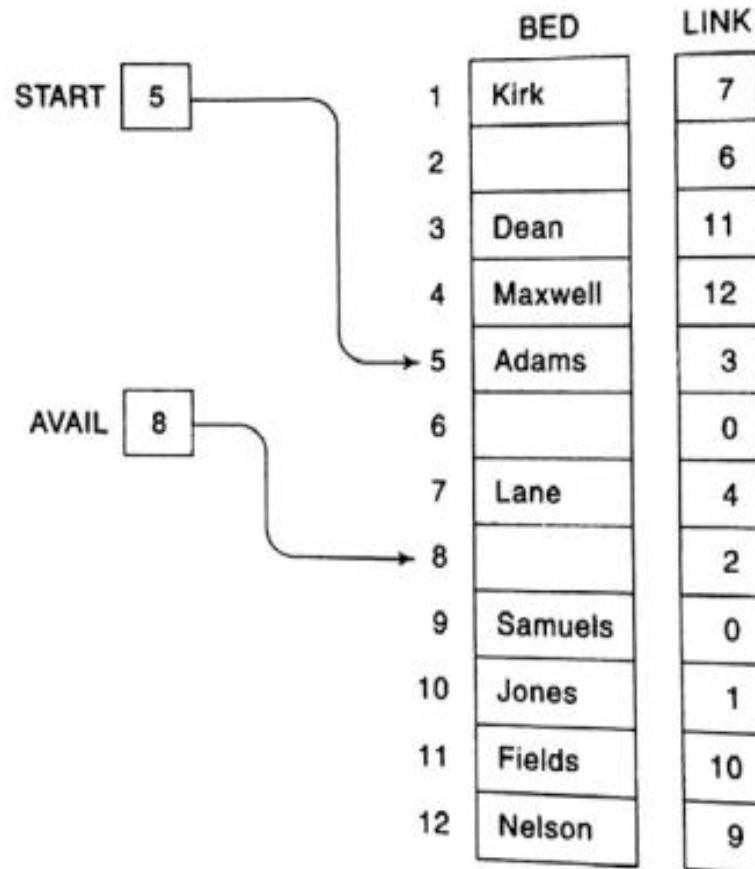


Fig. 5.28

Header Linked Lists

A *header linked list* is a linked list which always contains a special node, called the *header node*, at the beginning of the list. The following are two kinds of widely used header lists:

- (1) A *grounded header list* is a header list where the last node contains the null pointer. (The term “grounded” comes from the fact that many texts use the electrical ground symbol to indicate the null pointer.)
- (2) A *circular header list* is a header list where the last node points back to the header node.

Figure 5.29 contains schematic diagrams of these header lists. Unless otherwise stated or implied, our header lists will always be circular. Accordingly, in such a case, the header node also acts as a sentinel indicating the end of the list.

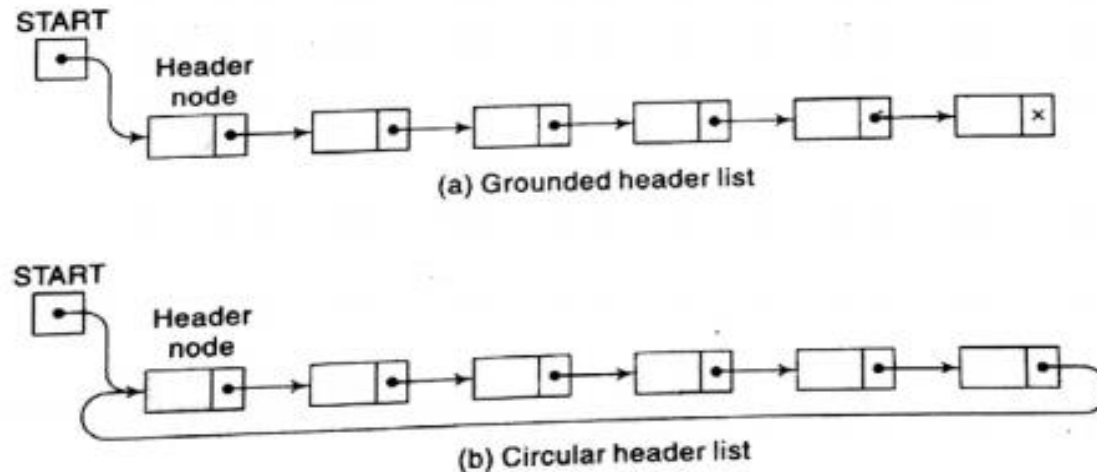


Fig. 5.29

Observe that the list pointer **START** always points to the header node. Accordingly, $\text{LINK}[\text{START}] = \text{NULL}$ indicates that a grounded header list is empty, and START indicates that a circular header list is empty.

Although our data may be maintained by header lists in memory, the **AVAIL** list will always be maintained as an ordinary linked list.

Header Linked Lists

- Circular header list are frequently used instead of ordinary linked list because many operation are much easier to implement header list. This comes from the following two properties, all circular header lists.
 - The NULL pointer is not used & hence contains valid address.
 - Every ordinary node has a predecessor. So the first node may not required a special case.

Remark: There are two other variations of linked lists which sometimes appear in the literature:

- (1) A linked list whose last node points back to the first node instead of containing the null pointer, called a *circular list*
- (2) A linked list which contains both a special header node at the beginning of the list and a special trailer node at the end of the list

Figure 5.31 contains schematic diagrams of these lists.

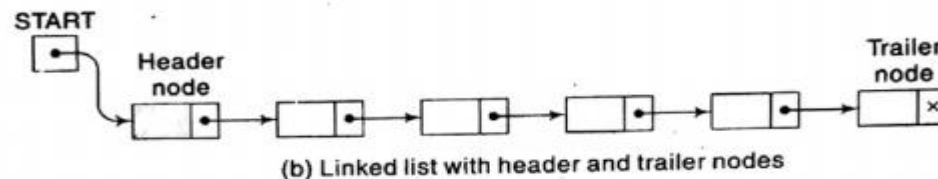
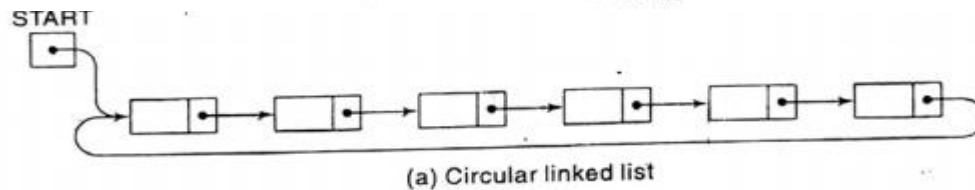


Fig. 5.31

Polynomials

Header linked lists are frequently used for maintaining polynomials in memory. The header node plays an important part in this representation, since it is needed to represent the zero polynomial. This representation of polynomials will be presented in the context of a specific example.

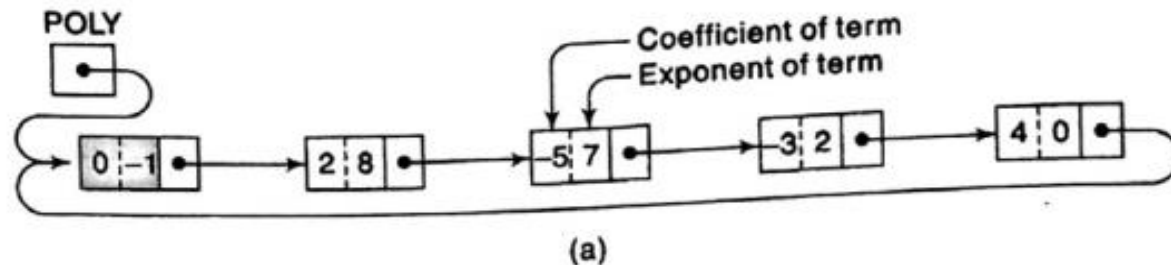
Example 5.20

Let $p(x)$ denote the following polynomial in one variable (containing four nonzero terms):

$$p(x) = 2x^8 - 5x^7 - 3x^2 + 4$$

Then $p(x)$ may be represented by the header list pictured in Fig. 5.32(a), where each node corresponds to a nonzero term of $p(x)$. Specifically, the information part of the node is divided into two fields representing, respectively, the coefficient and the exponent of the corresponding term, and the nodes are linked according to decreasing degree.

Observe that the list pointer variable POLY points to the header node, whose exponent field is assigned a negative number, in this case -1. Here the array representation of the list will require three linear arrays, which we will call COEF, EXP and LINK. One such representation appears in Fig. 5.32(b).



Two way Linked Lists

This section introduces a new list structure, called a two-way list, which can be traversed in two directions: in the usual forward direction from the beginning of the list to the end, or in the backward direction from the end of the list to the beginning of the list. Furthermore, given the location LOC of a node N in the list, one now has immediate access to both the next node and the preceding node in the list. This means, in particular, that one is able to delete N from the list without traversing any part of the list.

A *two-way list* is a linear collection of data elements, called *nodes*, where each node N is divided into three parts:

- (1) An information field INFO which contains the data of N
- (2) A pointer field FORW which contains the location of the next node in the list
- (3) A pointer field BACK which contains the location of the preceding node in the list

The list also requires two list pointer variables: FIRST, which points to the first node in the list, and LAST, which points to the last node in the list. Figure 5.33 contains a schematic diagram of such a list. Observe that the null pointer appears in the FORW field of the last node in the list and also in the BACK field of the first node in the list.

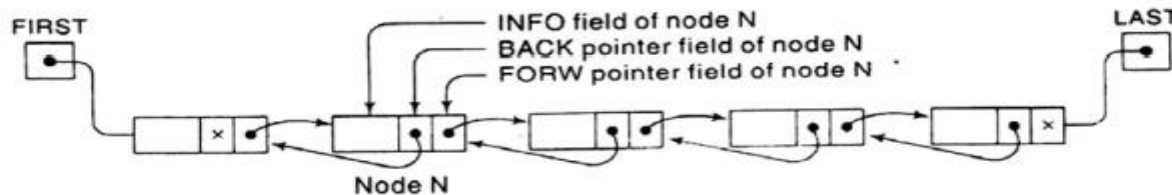


Fig. 5.33 Two-way List

Observe that, using the variable FIRST and the pointer field FORW, we can traverse a two-way list in the forward direction as before. On the other hand, using the variable LAST and the pointer field BACK, we can also traverse the list in the backward direction.

Suppose LOCA and LOCB are the locations, respectively, of nodes A and B in a two-way list. Then the way that the pointers FORW and BACK are defined gives us the following:

Pointer property: $\text{FORW}[\text{LOCA}] = \text{LOCB}$ if and only if $\text{BACK}[\text{LOCB}] = \text{LOCA}$

In other words, the statement that node B follows node A is equivalent to the statement that node A precedes node B.

Two way Linked Lists

Two-way lists may be maintained in memory by means of linear arrays in the same way as one-way lists except that now we require two pointer arrays, FORW and BACK, instead of one pointer array LINK, and we require two list pointer variables, FIRST and LAST, instead of one list pointer variable START. On the other hand, the list AVAIL of available space in the arrays will still be maintained as a one-way list—using FORW as the pointer field—since we delete and insert nodes only at the beginning of the AVAIL list.

Example 5.21

Consider again the data in Fig. 5.9, the 9 patients in a ward with 12 beds. Figure 5.34 shows how the alphabetical listing of the patients can be organized into a two-way list. Observe that the values of FIRST and the pointer field FORW are the same, respectively, as the values of START and the array LINK; hence the list can be traversed alphabetically as before. On the other hand, using LAST and the pointer array BACK, the list can also be traversed in reverse alphabetical order. That is, LAST points to Samuels, the pointer field BACK of Samuels points to Nelson, the pointer field BACK of Nelson points to Maxwell, and so on.

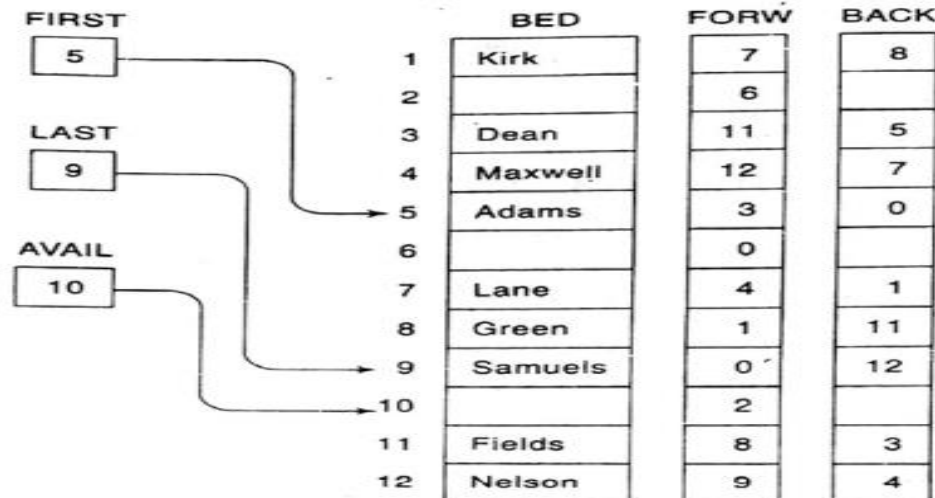


Fig. 5.34